
COMPLETE PROJECT HANDBOOK

DocuSage

An evidence-grounded PDF research assistant

Atif Saeed

Computer Science — AI engineering and retrieval systems

Architecture · Pipelines · Evidence verification · Evaluation

88-question reference appendix

Written from the DocuSage repository. Every threshold, limit and metric in this document is the real value from the real source.

How to use this handbook

This document is written so that you can explain DocuSage to anybody — a classmate, a professor, an interviewer, or a stranger at a demo table — without needing the code open in front of you.

Every chapter follows the same shape:

1. **Plain language first.** What the thing is and why it exists.
2. **Then the exact detail.** Real module names, real numbers, real file paths, so that when somebody asks "where is that in the code?" you can answer.

The last chapter is a **Question and Answer appendix** with more than eighty questions grouped by topic. If somebody puts you on the spot, that is where you look first.

Every fact in this handbook was read out of the repository as it stands today. Where a number appears (a threshold, a limit, a score), it is the real default from the real configuration file, not an approximation.

Table of contents

#	Chapter	What it answers
1	The thirty-second pitch	What is this project?
2	The problem it solves	Why does it need to exist?
3	What a user actually sees	What happens when I use it?
4	How to demo it in five minutes	What do I show, and in what order?
5	Architecture	How do the pieces fit together?
6	Tech stack, and why each piece	Why these tools and not others?
7	Repository map	Where does everything live?
8	Identity, quotas, and ownership	Who can do what, and how much?
9	Data stores	Where is the data, and in what shape?
10	Upload and indexing, A to Z	What happens to a PDF?
11	The ask-question pipeline, A to Z	What happens to a question?
12	Citations, claims, and PDF highlights	How is a citation proven?
13	The frontend workspace	How is the interface built?
14	API reference	What are the endpoints?
15	Memory, modes, and the LLM router	How is conversation and provider choice handled?
16	How quality is proven	How do you know it works?
17	Running it and deploying it	How do I start it?
18	Limits and honest answers	What does it not do?
19	File-by-file glossary	What is this file for?
20	Question and answer appendix	Everything else

1. The thirty-second pitch

DocuSage is a web application where you upload a PDF, ask questions about it in plain English, and get a structured answer where **every important claim carries a citation you can click**. Clicking the citation opens the original PDF inside the app and draws a highlight over the exact sentence the claim came from.

The line that matters most, and the one to lead with when explaining the project:

Most document chatbots treat the answer as the product. DocuSage treats verification as the product.

Anybody can wire a language model to a PDF and get an answer. The hard and interesting part — the part this project is actually about — is guaranteeing that the answer is traceable, and refusing to show a citation that cannot be proven.

If you want a single sentence for a resume or an elevator pitch:

DocuSage is a retrieval-augmented question answering system for PDFs where the model's output is treated as untrusted: quotes are verified against the retrieved source text, claims are re-anchored to the passage that actually supports them, and citations that cannot be located in the real document are dropped rather than displayed.

The one-paragraph technical version

A PDF is extracted page by page, split into structure-aware chunks, embedded with a small BGE embedding model, and stored in ChromaDB alongside a BM25 lexical index and a SQLite sidecar that records where every chunk's text sits on the page in PDF coordinates. A question is analysed for conversational references, optionally rewritten into a standalone query, then searched two ways at once — dense vectors and BM25 keywords — fused with Reciprocal Rank Fusion, reranked by a cross-encoder, and handed to a language model as a numbered evidence pool. The model must quote verbatim from that pool. Afterwards, a deterministic validation layer checks every quote against the retrieved chunk text, searches the whole recall pool for the passage that best supports each claim, maps the winning text back to real PDF bounding boxes, and decides for each citation whether the interface may show a precise highlight, a page-level reference, or nothing at all.

2. The problem it solves

The plain-language problem

Suppose you have a 300-page textbook, a company handbook, or a research paper, and you ask a chatbot a question about it. You get a confident, well-written answer. Now ask yourself the only question that matters:

Is it true?

To find out, you have to go and read the document. Which is the exact work you were trying to avoid. So the tool has not actually saved you anything — it has just moved the work and added a risk.

Why existing tools fall short

There are three failure patterns that show up over and over in "chat with your PDF" products.

One: the citation is too vague to check. The tool tells you "page 42". Page 42 has 600 words on it. Which sentence supports the claim? You still have to read the page and hunt for it.

Two: the citation is decorative. The tool shows you a chunk of retrieved text that is topically related to the answer, but the specific sentence the model claims to be quoting is not actually in there. The citation looks like evidence but is not evidence. This is worse than no citation, because it buys trust it has not earned.

Three: the tool would rather guess than refuse. Language models are trained to be helpful. Asked something the document does not cover, the path of least resistance is a plausible-sounding answer assembled from general world knowledge. For a study tool or a policy document this is the most dangerous behaviour possible, because the answer is fluent and wrong.

What DocuSage does about each one

Problem	What DocuSage does
Citation too vague	Resolves the quote to bounding-box coordinates on the page and draws a highlight over that specific text
Citation decorative	Verifies the quote actually appears in the retrieved chunk; strips the quote if it does not
Cites the wrong passage	Re-scores the claim against the whole recall pool and rebinds the citation to the passage that genuinely supports it
Guessing instead of refusing	Prompt explicitly prefers refusal; a separate verifier checks whether a refusal was itself wrong
Fake precision on scans	Scanned pages, figures, and tables are downgraded to page-level references instead of pretending to highlight a sentence

The unifying rule, and the thing to say if you only get one sentence about the design philosophy:

A citation that cannot be traced to real text on a real page is dropped rather than displayed.

Why refusing is a feature, not a limitation

This is worth rehearsing, because people often hear "it refuses to answer" as a weakness. Frame it the way the project does:

The value of DocuSage is that when it *does* answer, you can trust it. That guarantee only holds if the system is willing to say "I could not find that in the provided document." A system that always answers gives you no information about which answers to trust. A system that sometimes refuses tells you something real every time it does not.

3. What a user actually sees

This chapter is the user's experience in order, with no internals. If you are demoing to a non-technical audience, this chapter is your script.

3.1 Arriving

You open the app and you can use it immediately. There is **no login wall**. No email, no password, no credit card, no "start free trial" form.

Behind the scenes the browser generates a random session identifier and stores it locally, which is how the server recognises you on later requests. The user never sees this.

3.2 Uploading a PDF

The empty workspace shows a single call to action: *Upload a PDF to begin*. You can drag a file onto it or click to browse. The client checks three things before sending anything: that the file is a PDF, that it is not empty, and that it is within the size limit (25 MB by default).

3.3 Watching it get ready

Indexing is not instant, so the app does not pretend it is. Instead of a fake progress bar, the sidebar shows a checklist of real pipeline stages, and the document moves through them:

```
uploaded -> extracting -> chunking -> embedding -> indexing -> ready
                                                                |
                                                                (or failed)
```

Each stage is a real status written to the database by the indexing job, and the frontend polls for it. If a PDF has no extractable text — a pure scan, for example — the document ends in `failed` with a reason, and the interface offers a retry rather than silently producing a broken document you can ask questions about.

3.4 Asking a question

Once at least one document is `ready`, the chat interface appears. You type a question and the answer **streams in word by word**.

The answer is not a wall of prose. It is written as a short structured briefing — an overview, the key supported points, an example if the document contains one. Inside the text you see small superscript numbers: ¹, ², ³. Those are the citations.

3.5 Clicking a citation

This is the moment the whole project exists for, and the moment to slow down on in a demo.

Click a citation number and a panel slides over the screen showing the original PDF, scrolled to the cited page, with a **yellow highlight drawn over the exact sentence** that supports the claim. You can zoom, scroll to neighbouring pages, or open the PDF in a new browser tab at that page.

3.6 When the evidence is weaker

Not every citation can be a precise highlight, and the interface says so rather than faking it. Each citation carries a status badge:

What you see	What it means
Precise highlight	The exact supporting sentence was located on the page
Page reference	The right page was found, but not a tight sentence box — typical for scans, figures, and tables
Passage only	Supporting text is known, but it could not be placed on a page
Unavailable	No location information could be produced
Quote not verified	The model's quote did not appear in the source text, so it was removed

This honesty is a selling point, not an apology. Say it directly: the system tells you how strong each piece of evidence is, instead of making everything look equally certain.

3.7 Choosing what to search

A mode switcher above the composer offers two scopes:

- **All documents** — search everything you have uploaded.
- **One document** — restrict the search to a single selected file. Internally this is called **Super Focused** mode.

Super Focused is useful when you have several similar documents and want to be certain the answer came from a specific one. It is enforced in the backend, not just filtered in the interface: if Super Focused is on and no document is selected, the system refuses to search rather than quietly widening to your whole library.

3.8 When the document does not have the answer

You get an explicit statement that the information was not found in the provided document, and **no sources are shown**. Empty is the correct output here — a refusal with citations attached would imply evidence that does not exist.

3.9 Managing conversations

The sidebar keeps your chat history. You can rename a conversation, delete it, search across them, start a new one, and export a conversation to a Markdown file. Each answer also has copy and regenerate controls, and you can stop generation mid-stream — the partial answer is kept and marked as stopped rather than being thrown away.

3.10 Hitting the trial limit

The guest trial covers **one PDF and fifteen questions**. When you run out, a sign-in prompt appears for the first time. Signing in with Google is also free and raises the limits to **five PDFs and one hundred questions per calendar month**.

The important detail, and a good one to mention because it shows the design was thought through: signing in **carries your trial work over**. The PDF you already uploaded and the conversations you already had are moved to your new account. You do not start again.

Equally important, the questions you already asked during the trial are counted against your first month. Signing in repeatedly cannot be used to reset the counter.

4. How to demo it in five minutes

A tested running order. The goal is to reach the highlight moment fast, because that is the part people remember.

Before you start

- Backend running on port 8000, frontend on port 3000.
- Have a PDF ready with **real text**, not a scan. A textbook chapter, a policy handbook, or a paper works well.
- Know one question the document definitely answers, and one it definitely does not. You will use both.

Minute 1 — Frame the problem

Do not open the app yet. Say this first:

"Every tool like this gives you an answer. The question nobody answers is whether the answer is true. Watch what happens when I click a citation."

Setting the expectation before the demo is what makes the payoff land.

Minute 2 — Upload

Drag the PDF in. While it indexes, narrate the checklist:

"It is extracting the text, splitting it into chunks, embedding them into a vector store, and building a keyword index. It is also recording where every chunk sits on the page — that last part is what makes the highlight possible later."

Minute 3 — Ask the question you know it answers

Ask your prepared question. Let the answer stream. Point at the structure:

"Notice it is a briefing, not an essay. And notice the numbers in the text."

Minute 4 — The payoff

Click a citation. Wait for the panel. Then say:

"That is the original PDF, on the right page, with the sentence highlighted. I did not tag that by hand. The system resolved the model's quote back to coordinates on the page and verified the text was really there."

This is the moment. Give it a beat before moving on.

Minute 5 — Now show the refusal

Ask the question you know the document does not cover.

"It says it could not find that, and it shows no sources. That is the whole point. A tool that always answers tells you nothing about which answers to trust."

If you have extra time

Pick whichever suits the audience:

- **Super Focused mode** — switch to one document and show the scope change.
- **A scanned PDF** — show the page-level badge and explain that the system refuses to fake a sentence-level highlight.
- **The quota flow** — hit the trial limit and show that sign-in carries your work across instead of resetting it.
- **The evaluation results** — open `evaluation/results/acceptance_latest.json` and show that the quality bars are machine-checked, not claimed.

What not to do

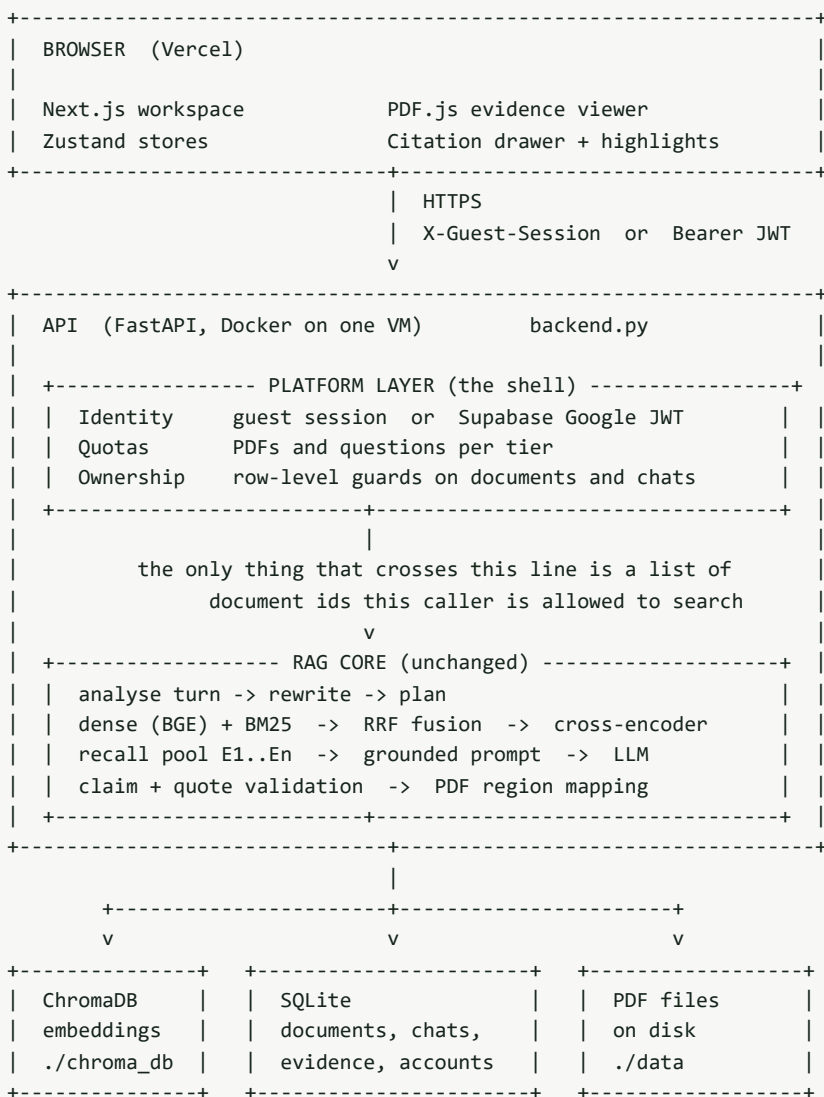
- Do not open with the architecture diagram. Show the product first.
 - Do not use a scanned PDF for the main demo; you will not get the highlight.
 - Do not skip the refusal. It looks like a limitation if unexplained and like rigour if explained.
-

5. Architecture

5.1 The shape in one sentence

A Next.js frontend talks to a FastAPI backend; the backend resolves *who you are* and *what you are allowed to touch* before any retrieval happens; retrieval and generation then run over ChromaDB, a BM25 index, and SQLite, and everything the model produces is validated before it reaches the screen.

5.2 The layer diagram



5.3 The single most important architectural decision

The platform layer is a thin shell around an unchanged retrieval pipeline.

Identity, quotas, and ownership are all resolved *in the API*, before `rag.ask_question` is ever called. User isolation works by filtering the list of document identifiers that reaches retrieval. There is **no isolation logic anywhere inside the RAG core**.

Why this matters, and why it is the right answer to "how do you keep multi-tenancy from leaking?":

- **Retrieval cannot leak, because it never sees a forbidden identifier.** The guard removes it upstream. There is no code path where retrieval has to remember to check.
- **The RAG pipeline stays testable in isolation.** You can benchmark retrieval quality without constructing users, sessions, or tokens.
- **Swapping the identity provider is a one-file change.** Only `app_platform/auth/supabase_jwt.py` knows what a Supabase token is.
- **Swapping the database is a URL change plus one file.** Only `database/connection.py` knows where the relational store lives.

5.4 Request flow, end to end

```
1. Browser sends the question, plus a session header or bearer token
2. get_request_context -> RequestContext(actor_type, actor_id)
3. ownership.assert_conversation_owner (403 if someone else's chat)
4. quotas.check_question_allowed (402 if out of allowance)
5. ownership.visible_document_ids (drop foreign document ids)
6. resolve_retrieval_scope (only status='ready' documents)
7. rag.ask_question(generate=False) (retrieve, rerank, build prompt)
8. yield __CITATIONS__ frame (interface can render sources now)
9. stream LLM tokens (resolving markers as they arrive)
10. verify_and_repair_refusal (was the refusal itself wrong?)
11. finalize_answer_citations (verify quotes, rebind, map to PDF)
12. visible_sources (decide what the UI may show)
13. yield __CITATIONS_FINAL__ frame (enriched, verified citations)
14. save the message with its citations
```

Two things about that ordering are deliberate and worth pointing out:

Quota and ownership rejections happen before the stream opens. Once a streaming response has started, the HTTP status code is already sent as 200. So the checks run first, which means a blocked request gets a real 402 or 403 status code the client can act on, instead of an error message buried inside a successful-looking response body.

Citations are sent before the answer text. The interface can render the source list immediately, then fill in the answer as tokens arrive. A second, enriched citation frame is sent at the end once validation has run.

5.5 Two calls to the model, at most — and never two retrievals

Retrieval runs **once** per question. The streaming endpoint calls `ask_question(..., generate=False)`, which does all the retrieval work and builds the prompt but does not generate; the endpoint then streams the

generation itself. This avoids the obvious bug of retrieving twice, once for the prompt and once for the answer.

A second model call happens only in one situation: the model refused, but the retrieved passages actually did contain support. That triggers one constrained retry. Even then, retrieval is not repeated — the same prompt and the same evidence are reused.

6. Tech stack, and why each piece

The table first, then the reasoning. If you are asked "why did you choose X?", the reasoning section is the answer — and note that "it is free and it is small enough to run on one cheap virtual machine" is a legitimate engineering answer, not an excuse.

6.1 The stack

Layer	Choice	Version
Frontend framework	Next.js, App Router	14.2.35
UI library	React	18
Language	TypeScript	5, strict
Styling	Tailwind CSS with custom design tokens	3.4
Client state	Zustand	5
PDF rendering	pdfjs-dist	4.10.38
Markdown rendering	react-markdown, remark-gfm, rehype-sanitize	10 / 4 / 6
Icons	lucide-react	1.28
Frontend tests	Vitest	4
API framework	FastAPI with Uvicorn	—
Vector store	ChromaDB, persistent client	—
Embeddings	BAAI/bge-small-en-v1.5 via FastEmbed	—
Lexical search	BM25 Okapi via rank_bm25	—
Fusion	Reciprocal Rank Fusion	—
Reranker	BAAI/bge-reranker-base via FastEmbed ONNX	—
PDF extraction	PyMuPDF primary, pypdf fallback	—
Chunking	langchain-text-splitters	—
Generation	Groq primary, four fallbacks, Ollama for local	—
Auth	Supabase, Google OAuth only	JS client 2.115
Relational store	SQLite	—
Hosting	Vercel for frontend, Docker on one VM for API	—

6.2 Why each choice

Next.js App Router. The product is a single workspace screen, so routing is not the draw. What is useful is server-side rendering for the initial shell, the built-in font optimisation, and one-click deployment to Vercel.

Zustand rather than Redux or React Context. There are four independent concerns — documents, chat, auth, toasts — and they are read by components at very different depths of the tree. Zustand gives selector-based subscriptions, so a component that only reads the document list does not re-render when a chat token arrives. During streaming, tokens arrive many times per second, which makes that distinction matter. Context would re-render every consumer on every update.

PDF.js. It is the only realistic option for rendering a PDF in a browser canvas with control over per-page coordinates. Since the highlight overlay requires mapping PDF-space bounding boxes onto rendered canvas pixels, that control is essential rather than nice to have.

FastAPI. Type-hint-driven request validation through Pydantic, native support for streaming responses, and a dependency injection system that makes the identity resolution a clean one-line dependency on every route (`context: RequestContext = Depends(get_request_context)`).

ChromaDB. Runs embedded, persists to a local directory, requires no separate server process. For a single-VM deployment with a handful of documents per user that is exactly right. A hosted vector database would add cost, latency, and an external dependency for no benefit at this scale.

BAAI/bge-small-en-v1.5. The small variant is a deliberate trade. It is around 33 million parameters, runs on CPU in a few hundred milliseconds, and has a roughly 512-token window that the 1200-character chunk size was chosen to fit comfortably inside. A larger embedder would improve recall slightly and cost a great deal more memory on a free-tier VM. More importantly, the architecture compensates for embedding weakness in two later stages: BM25 catches exact terms that dense vectors blur, and the cross-encoder reranks whatever the first stage retrieves. Spending the compute budget on reranking rather than on a bigger embedder is the better trade at this scale.

BM25 alongside dense retrieval. Embeddings are good at meaning and bad at exact strings. Ask for "Article 5" or "Figure 3" or a specific technical term and a dense vector will happily return something semantically adjacent instead. BM25 is the opposite: it is precise about tokens and blind to meaning. Running both and fusing them covers both failure modes.

Reciprocal Rank Fusion. It combines two ranked lists using only the ranks, not the scores. This matters because a cosine distance and a BM25 score are not on comparable scales, and normalising them against each other would require tuning that breaks whenever the corpus changes. RRF sidesteps the problem entirely.

A cross-encoder reranker. The first retrieval stage scores the query and the passage separately and compares the results. A cross-encoder reads both together, which is far more accurate and far too slow to run over an entire corpus. So it runs over roughly twenty fused candidates, which is affordable. This is the standard retrieve-then-rerank pattern and it is where most of the quality comes from.

Groq for generation. Very fast inference on Llama models with a free tier, which suits a streaming interface where perceived latency is what the user notices. Four fallback providers exist so a single provider

outage does not take the product down.

Supabase for auth, Google only. Delegating OAuth means never storing a password. Restricting to Google keeps the sign-in screen to one button. The backend only ever verifies a JWT signature, which is why replacing this provider touches exactly one file.

SQLite. No server to run, no connection pool to manage, and it is a single file that Docker can bind-mount so a container rebuild does not lose user data. The abstraction is already in place for Postgres: change `DATABASE_URL` and extend `database/connection.py`.

7. Repository map

7.1 Top level

backend.py	FastAPI routes, streaming protocol, request wiring
rag.py	Retrieval and generation orchestration
config.py	Every retrieval and generation tuning value
app.py	Legacy Streamlit entrypoint (NOT the product)
app_platform/ settings.py	Identity, quotas, ownership guards Env-driven deployment configuration
auth/ quotas/ guards/ownership.py	Guest sessions and Supabase JWT verification Per-tier limits and usage recording Row-level access control
database/ memory/ llm/ evaluation/ frontend/ components/ docs/	SQLite stores and migrations Conversation history Provider router and per-provider clients Benchmarks, adversarial pack, acceptance gates The Next.js application (the real product UI) Legacy Streamlit UI components (NOT the product) This handbook

7.2 The retrieval and evidence modules

These are the heart of the project. Grouped by what they do rather than alphabetically, because that is how you will need to explain them.

Getting text out of a PDF and into the indexes

File	Role
pdf_extraction.py	PyMuPDF primary, pypdf fallback below 50 characters
chunking.py	Structure-aware chunking with heading detection
indexer.py	Orchestrates the whole indexing job
evidence_mapping.py	Maps chunk text to PDF bounding boxes at index time
index_hygiene.py	Keeps Chroma, BM25, SQLite, and disk consistent

Finding the right passages

File	Role
<code>hybrid_retrieval.py</code>	Dense plus BM25, RRF fusion, rerank orchestration
<code>bm25_index.py</code>	BM25 index built over the same Chroma corpus
<code>reranker.py</code>	Cross-encoder scoring and final evidence selection
<code>rerank_calibration.py</code>	Converts raw logits into relevance percentages
<code>query_retrieval.py</code>	Query-type profiles: figures, articles, front matter, listings

Understanding the question

File	Role
<code>conversation_query.py</code>	Deterministic turn analysis: intent, relation, references
<code>query_rewriter.py</code>	LLM rewrite into a standalone query, only when needed
<code>answer_planner.py</code>	Maps intent to briefing components and sub-queries

Building the prompt and generating

File	Role
<code>answer_prompt.py</code>	The grounding contract and evidence formatting
<code>evidence_focus.py</code>	Passage role labelling and citation allowlist
<code>llm_service.py</code>	Stable generation facade used by RAG code
<code>llm/router.py</code>	Provider chain with fallback rules

Proving the answer

File	Role
<code>citation_resolver.py</code>	Parses and validates <code>[E#]</code> markers, including mid-stream
<code>claim_validator.py</code>	The finalisation pipeline; verifies quotes against chunks
<code>claim_localizer.py</code>	Claim to source span to PDF region localisation
<code>claim_orchestrator.py</code>	Rebinds a claim to the passage that actually supports it
<code>claim_reanchor.py</code>	Neighbour stitching when a claim spans a chunk boundary
<code>quote_evidence.py</code>	Query-time quote to PDF span mapping, with cache
<code>visual_evidence.py</code>	Figure, table, and scan policy — never fake precision
<code>grounding_verifier.py</code>	Detects and repairs a wrong refusal
<code>evidence_state.py</code>	Citeable versus page-only, and what the UI may show
<code>evidence_trace.py</code>	Structured observability for the whole chain
<code>answer_formatter.py</code>	Removes prose quote dumps, caps marker count
<code>response_validator.py</code>	Gate before an answer is persisted

7.3 The Streamlit files are not the product

This trips people up when reading the repository, so address it directly.

`app.py` and the `components/` directory are a **Streamlit prototype** from earlier in the project's life. The real product interface is the Next.js application in `frontend/`.

The honest framing when asked: the project started as a Streamlit prototype because that was the fastest way to validate that the retrieval pipeline produced useful answers. Once the pipeline was working, the interface moved to Next.js because Streamlit cannot render a PDF with coordinate-accurate highlight overlays — which is the core feature. The prototype was left in the repository rather than deleted; it is not wired into the deployment.

The git history shows this arc across thirty-one commits: an initial Streamlit knowledge assistant, then a Next.js workspace, then reranking, then three iterations of evidence-based highlighted citations, then the platform layer for guest trials and Google auth.

8. Identity, quotas, and ownership

This is the `app_platform/` package: the shell that decides who you are, whether you are allowed to do the thing, and whether the row you asked for is yours.

8.1 One actor per request

Every request resolves to exactly one **actor**, which is either a guest session or a signed-in user. The dataclass is deliberately tiny:

```
RequestContext(actor_type, actor_id, email)
    actor_type is "guest" or "user"
    tier is "guest" or "free"
```

Ownership rows and usage counters are both keyed by `(actor_type, actor_id)`, which is what allows the two tiers to share every piece of downstream code. There is no separate guest code path and no separate user code path.

8.2 How identity is resolved

From `app_platform/auth/dependency.py`:

```
Authorization: Bearer <jwt>    present and valid -> user actor
                                         (verify signature,
                                         upsert the profile)
X-Guest-Session: <uuid>        present           -> guest actor
                                         (create session if new)
neither                        -> 401, reload to start trial
```

A valid bearer token always wins over a guest header. This handles the real situation where a user signs in but their browser still holds the trial identifier from before. Without this rule the same person could be treated as two different actors depending on request ordering.

Token verification is HS256 against `SUPABASE_JWT_SECRET`, in `app_platform/auth/supabase_jwt.py`. If `AUTH_PROVIDER` is not `supabase`, presenting a token is an error rather than being silently ignored — the server says sign-in is not enabled here.

8.3 The limits

Tier	PDFs	Questions	Persistence
Guest	1	15 for the entire trial	Session only
Signed in, free	5	100 per calendar month	Saved to the account

Upload size limit: 25 MB, both tiers.

Every one of those numbers is an environment variable read at request time: `QUOTA_GUEST_MAX_PDFS` , `QUOTA_GUEST_MAX_QUESTIONS` , `QUOTA_USER_MAX_PDFS` , `QUOTA_USER_MAX_QUESTIONS_MONTHLY` , `QUOTA_MAX_PDF_MB` . Because they are read at request time rather than at import time, retuning the product means editing `.env` and restarting — no code change, no redeploy of new logic.

8.4 Where quota checks run, and why there

From the module docstring of `app_platform/quotas/service.py` :

Checks run in the API shell before any retrieval work starts, so a blocked request never reaches the RAG pipeline or the LLM provider.

That is the design rule. A user who is out of allowance costs zero embedding compute and zero LLM tokens.

Two details worth knowing because they are the kind of thing an interviewer probes for:

PDF usage is a live count, not a counter. `pdfs_used` runs `count_documents_for_owner` against the database rather than reading an incrementing integer. Deleting a document therefore frees a slot immediately. A stored counter would have drifted out of sync with reality on every delete.

Questions are counted after retrieval succeeds, not when the request arrives. In the streaming endpoint, `quotas.record_question` is called after `ask_question` has returned successfully. A request that fails during retrieval does not consume the user's allowance.

Regeneration is free. `if not request.regenerate: quotas.check_question_allowed(...)` — the user already paid for that answer once.

8.5 Ownership: knowing a UUID is not enough

From `app_platform/guards/ownership.py` :

Knowing a UUID must not be enough to read someone else's document or chat.

The guards are `assert_document_owner` , `assert_conversation_owner` , and `visible_document_ids` . The status codes are chosen carefully:

Situation	Response
Row does not exist	404 Document not found
Row exists, belongs to another actor	403 with the message "Document not found."
Conversation does not exist yet	Allowed — creation binds it to this actor

Notice that the 403 message deliberately says *not found* rather than *not allowed*. Saying "you are not allowed to see this" confirms the identifier is real, which leaks information. The status code differs for the server's own diagnostics; the message the user sees does not.

Legacy rows with a NULL owner stay readable. This is a documented, intentional decision: it means upgrading an existing installation does not orphan data that was created before ownership columns existed.

`visible_document_ids` is applied **twice** in the chat path — once to what the client asked for, and again to whatever the retrieval scope resolved. Belt and braces, so no foreign document identifier can reach the pipeline by any route.

8.6 Asking about someone else's documents

An interesting edge case handled explicitly in `backend.py` :

```
# Asking only about someone else's documents searches nothing, rather
# than silently widening to the whole corpus.
if requested and not visible_requested:
    return None, insufficient_context_payload()
```

If you request only document identifiers you do not own, the filtered list is empty. The naive behaviour would be to treat "no documents specified" as "search everything" — which would turn a failed access attempt into a successful search of your own library, producing a confusing answer. Instead it searches nothing and returns the insufficient-context response.

8.7 Guest to account migration

`POST /auth/migrate-guest` is called by the frontend on first successful sign-in. The signed-in token identifies the destination; the `X-Guest-Session` header names the trial to claim.

```
1. Caller must be a signed-in user          (else 401)
2. Look up the guest session
3. If missing or already migrated          -> return already_migrated
4. reassign_documents      guest -> user
5. reassign_conversations  guest -> user
6. mark_guest_migrated     (a trial can only be claimed once)
7. Replay the trial question count against the new monthly allowance
```

Step 7 is the anti-abuse measure, and the code says so:

```
# Trial questions already asked count against the new monthly allowance,  
# so signing in cannot be used to reset the counter repeatedly.
```

8.8 Error contract

Code	Meaning	When
401	Authentication required	Missing session, expired token, sign-in disabled
402	QUOTA_EXCEEDED	Out of PDFs or questions; body carries resource, limit, used, and an upgrade hint
403	Forbidden	Row belongs to another actor
404	Not found	Row genuinely does not exist
413	Payload too large	PDF above the size limit

The 402 body includes an `upgrade_hint` field whose value is `sign_in`, `delete_document`, or `wait_for_reset`. The frontend uses it to decide whether to show the sign-in modal or a different message, so the interface never has to guess by parsing prose.

9. Data stores

Three stores, each holding what it is good at.

+-----+	+-----+	+-----+
ChromaDB	SQLite	Filesystem
./chroma_db	./data/documents.db	./data
chunk text	metadata, chats,	original PDFs
384-d vectors	evidence sidecar,	
chunk metadata	accounts, usage	
+-----+	+-----+	+-----+

9.1 ChromaDB

A `PersistentClient` at `CHROMA_DB_PATH` (default `./chroma_db`), with a single collection named by `COLLECTION_NAME` (default `ml_notes` — a leftover from the project's earliest days, when it indexed machine-learning lecture notes).

Each record holds the chunk text, its embedding vector, and metadata: `document_id`, `filename`, `page_start`, `page_end`, `section_title`, `section_id`, `prev_chunk_id`, `next_chunk_id`.

Chunk identifiers are `{document_id}_{index}` — deterministic, so re-indexing the same document produces the same identifiers, and the neighbour links needed for cross-chunk re-anchoring are trivial to compute.

9.2 SQLite tables

Nine tables. Created by `init_db()` in `database/db.py` plus the migration in `database/migrations/001_platform.sql`.

documents — the registry that decides what is searchable.

```
document_id TEXT PRIMARY KEY, filename, upload_time,
total_pages, total_chunks, status DEFAULT 'uploaded', index_error,
owner_type, owner_id
```

conversations — `conversation_id`, `title`, `created_at`, `updated_at`, `owner_type`, `owner_id`.

messages — `id`, `conversation_id`, `role`, `content`, `citations` (JSON text), `created_at`. Indexed on `(conversation_id, id)`.

Storing the citations as JSON alongside the message is what allows a reloaded conversation to still have working, clickable citations rather than plain text with dead superscripts.

page_layouts — the geometry of every page.

```
document_id, page_number, width, height, source, engine, span_count
PRIMARY KEY (document_id, page_number)
```

Needed because a highlight is drawn in PDF points but rendered on a canvas scaled to the browser viewport. Without the page dimensions the overlay cannot be positioned.

chunk_evidence — the index-time provenance sidecar, and arguably the most distinctive table in the schema.

```
chunk_id TEXT PRIMARY KEY, document_id, page_start, page_end, snippet,
highlight_available INTEGER, match_type, source, text_engine, layout_engine,
join_recovered INTEGER, ranges_json, regions_json, segments_json
```

regions_json holds the actual bounding boxes. **match_type** records *how* the chunk text was aligned to the page spans — exact, normalized, fuzzy_compact, or hyphen_fuzzy — which is how the system knows whether to trust the geometry.

quote_region_cache — memoised quote-to-coordinate lookups.

```
chunk_id, quote_hash, document_id, quote_text, match_type,
quote_highlight_available, regions_json, page_start, page_end, created_at
PRIMARY KEY (chunk_id, quote_hash)
```

Resolving a quote to coordinates involves fuzzy alignment over page spans. The same quote gets resolved repeatedly — once during answer finalisation, again when the user opens the drawer, again if they reopen it. The cache turns that into one computation.

users — **user_id**, **auth_subject** (unique), **email**, **created_at**, **last_seen_at**. No passwords, ever; **auth_subject** is the Supabase subject claim.

guest_sessions — **session_id**, **created_at**, **last_seen_at**, **pdf_count**, **question_count**, **migrated_to_user_id**. That last column is what makes a trial claimable exactly once.

usage_counters — (**actor_type**, **actor_id**, **period**) as the primary key, plus **question_count** and **updated_at**. Guests use the literal period **trial**; users use a calendar month string. One table serves both tiers.

9.3 Migrations under SQLite's constraints

SQLite has no **ADD COLUMN IF NOT EXISTS**, so the pattern throughout is a guarded **ALTER TABLE** that swallows the resulting error:

```
try:
    cursor.execute("ALTER TABLE documents ADD COLUMN index_error TEXT")
except sqlite3.OperationalError:
    pass
```

Ugly, but idempotent and honest about the constraint. Alongside it, `_run_sql_migrations` applies `.sql` files from the migrations directory in name order, and each file is written to be idempotent.

9.4 Index hygiene: the three-store consistency problem

Three stores holding facets of the same object means three ways to drift apart. A deleted document could leave orphaned vectors in Chroma, stale terms in BM25, or abandoned bounding boxes in SQLite. Orphaned vectors are the dangerous case, because they can surface as an answer citing a document the user believes they deleted.

`index_hygiene.py` states the rule:

Live retrieval may only search documents that exist in SQLite with status `ready`. Deletes and re-indexes must purge Chroma, BM25, evidence sidecars, and on-disk PDFs so orphan chunks cannot leak into library answers.

SQLite is the authority. `live_searchable_document_ids()` returns the ready document identifiers, and retrieval receives an explicit list. The critical consequence, spelled out in the docstring:

Normal mode always returns an explicit list (possibly empty). Empty means search nothing, never "all of Chroma".

`reconcile_index()` runs at startup to sweep up anything that drifted while the process was down — for example if the container was killed mid-delete.

9.5 What lives where, and what is lost if you delete it

Delete this	You lose	Recoverable?
<code>chroma_db/</code>	Embeddings and the searchable chunk text	Yes, by re-indexing the PDFs
<code>data/*.pdf</code>	The original files	No
<code>data/documents.db</code>	Metadata, chats, evidence, accounts	No
BM25 index	Nothing — it is in-memory and rebuilds itself	Automatically

This is exactly why the Docker Compose file bind-mounts `./data` and `./chroma_db` from the host: a container rebuild must not lose user data. The model cache is a named volume for the same reason, since first boot otherwise re-downloads the embedding and reranker weights.

10. Upload and indexing, A to Z

What happens between dropping a PDF on the page and being able to ask questions about it.

10.1 The pipeline

```
POST /upload
|
|-- check quota      (PDF slots available?)          -> 402 if not
|-- check size      (within QUOTA_MAX_PDF_MB?)      -> 413 if not
|-- create_document (row with status='uploaded', bound to this actor)
|-- write the file   ./data/{document_id}.pdf
|-- schedule index_pdf as a FastAPI background task
|
+-- return { document_id, status: "uploaded" } <-- returns immediately

    then, in the background:

status='extracting'  PyMuPDF page by page; pypdf if under 50 chars
status='chunking'    structure-aware chunks, 1200 chars, 150 overlap
                    build evidence regions -> chunk_evidence, page_layouts
status='embedding'   BAAI/bge-small-en-v1.5, batches of 50
status='indexing'    write to Chroma, invalidate the BM25 cache
status='ready'       searchable
status='failed'      with index_error if no text could be extracted
```

The upload response returns before indexing finishes. This is deliberate: indexing a large PDF takes tens of seconds, and holding an HTTP request open that long invites proxy timeouts. Instead the client receives a document identifier immediately and polls `GET /documents/{id}` for the status, which is what drives the checklist in the interface.

10.2 Extraction, with a fallback

`pdf_extraction.py` uses **PyMuPDF** as the primary engine and **pypdf** as a fallback, triggered when the primary produces fewer than `MIN_PRIMARY_CHARS = 50` stripped characters in total.

Two engines because they fail differently. PyMuPDF is faster and gives span-level geometry, which is required for highlights. But some PDFs — unusual generators, certain encodings — yield little or nothing from it while pypdf reads them fine. Trying the second engine costs nothing when the first succeeds.

Both read **page by page from disk** rather than loading the whole file into memory. On a 12 GB VM serving several users, holding an entire large PDF in Python memory is a real risk.

The result carries diagnostics: which engine was used, how many pages had text, character counts per engine, and whether the fallback was tried. When a document fails, these are what tell you why.

10.3 Chunking

From `chunking.py` :

Parameter	Value	Reasoning
<code>CHUNK_SIZE</code>	1200 characters	Comfortably inside the roughly 512-token window of bge-small
<code>CHUNK_OVERLAP</code>	150 characters	A sentence straddling a boundary survives in one piece somewhere
<code>MIN_CHUNK_CHARS</code>	200	Below this, merge — a 40-character fragment is not retrievable context
<code>PAGE_JOIN</code>	<code>\n\n</code>	Pages are concatenated, so a chunk may span a page break

The comment in the source explains the choice against the earlier version:

Parameters chosen for bge-small-en-v1.5 (~512 token limit): ~1200 chars stays comfortably under the embedding window while giving more context than the V1 page-isolated 800/100 setup.

Chunks are built from the whole document, not per page. This is the important difference from the naive approach. If a definition begins at the bottom of page 7 and finishes at the top of page 8, page-isolated chunking splits it and neither half retrieves well. Document-level chunking keeps it together, and page boundaries are preserved as `page_start` and `page_end` metadata instead of being enforced as hard splits.

Headings are detected deterministically — no LLM call. Three conservative patterns: `UNIT I` style markers, numbered sections such as `1.2` or `2.5.1`, and short all-capitals title lines. A detected heading contributes `section_title` and a slugified `section_id` to every chunk beneath it, which gives retrieval and the citation UI a human-readable location.

Splitting itself uses `RecursiveCharacterTextSplitter` with a separator ladder that tries to break at the most semantically natural point available:

```
"\n\n" -> "\n" -> ". " -> "? " -> "! " -> "; " -> ", " -> " " -> ""
```

10.4 Evidence mapping: where the highlight comes from

This is the step that makes the product's headline feature possible, and it runs at **index time**, not at question time. `evidence_mapping.py` takes each chunk's text and works out which spans on which pages it came from, recording their bounding boxes.

The docstring calls the algorithm the mapping proof, and it is a four-level cascade:

1. Exact substring of the concatenated span text
2. Whitespace-normalised compact comparison with an index map
3. Compact substring search
4. Hyphen-stripped compact search

Each level is more forgiving than the last, because PDF text extraction is messy. It contains non-breaking spaces, typographic quotes, ligatures, and hyphens inserted for line breaking that are not in the logical word. Level 1 handles clean documents; the later levels recover the rest.

There is also a recovery path for chunks that span a page join. Since pages were concatenated with `\n\n`, a chunk crossing that boundary is not a substring of any single page. Those chunks are split on the join and each part is mapped independently.

Two rules from the docstring are worth quoting verbatim, because they are the integrity guarantee of the whole feature:

Coordinates are PyMuPDF page space (origin top-left, PDF points).

Never invent boxes: `highlight_available` is true only when every non-empty slice mapped to at least one real span bbox.

That flag is the honest signal that flows all the way through to the badge the user sees on a citation.

10.5 Embedding and writing to the indexes

Embedding uses `TextEmbedding("BAAI/bge-small-en-v1.5")` from FastEmbed, in **batches of 50** with explicit garbage collection between batches. On a small VM this is what keeps memory flat on a large document rather than growing until the process is killed.

Before writing, `purge_chroma_document` removes any existing vectors for that document identifier. This makes re-indexing idempotent: you never end up with two generations of chunks for the same document, which would produce duplicate citations.

Finally `bm25_index.invalidate()` is called. BM25 is built lazily from the Chroma corpus and cached with a fingerprint, so invalidating means the next query rebuilds it including the new document.

10.6 Failure handling

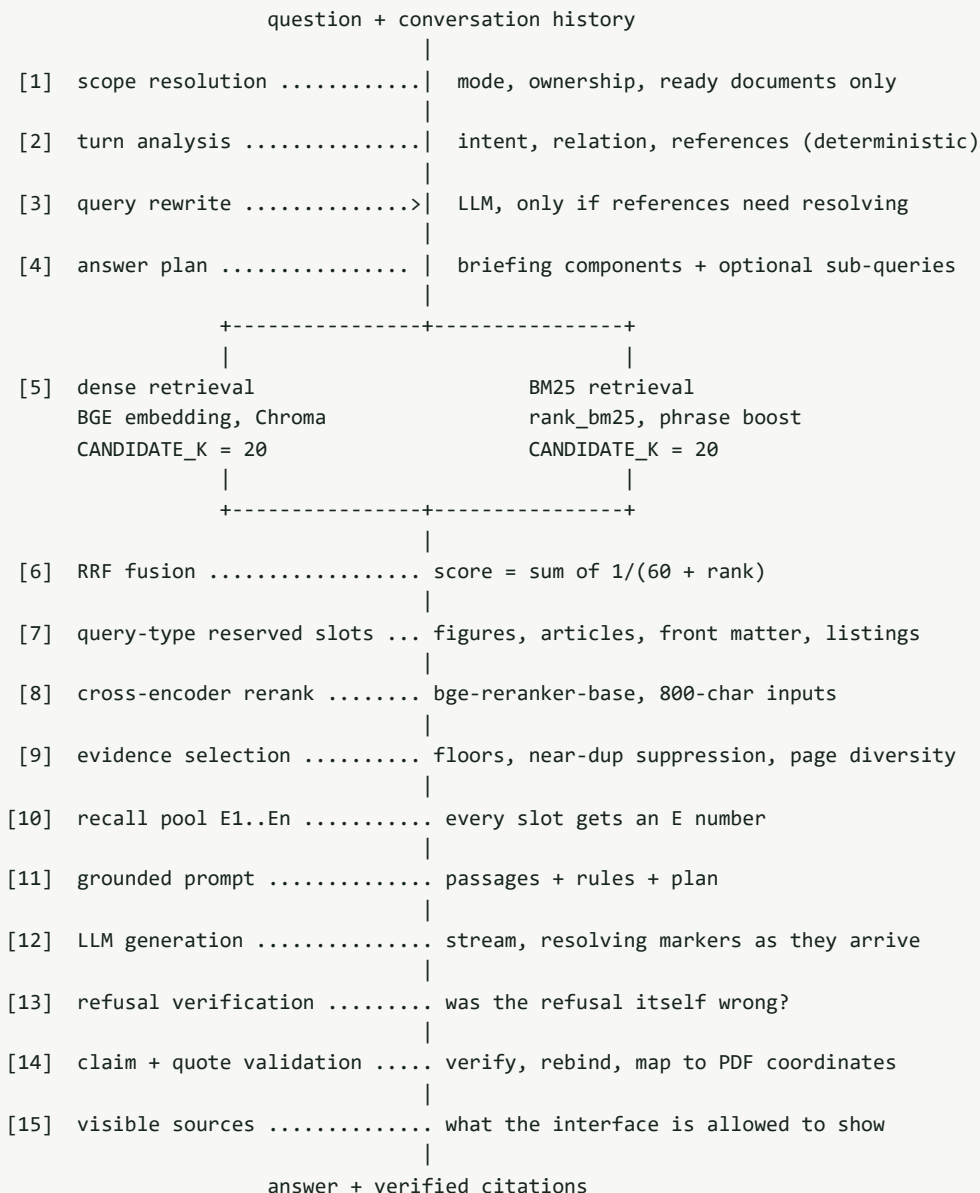
Failure	Behaviour
No text extracted	<code>purge_document_index</code> , status <code>failed</code> , <code>index_error = INDEX_FAILURE_NO_TEXT</code> , logged with the engine and page counts
Evidence mapping fails	Non-fatal. The document still indexes and answers; citations degrade to page level
Chroma write fails	Status <code>failed</code> , error recorded

That middle row is a deliberate degradation choice worth explaining: highlights are the premium feature, but a document you can ask questions about with page-level citations is far more useful than a document that refused to index because its geometry was unusual. The feature degrades; the product does not break.

11. The ask-question pipeline, A to Z

The central technical chapter. This follows `ask_question` in `rag.py` and the streaming endpoint in `backend.py`, in execution order.

11.1 The whole pipeline at a glance



11.2 Step 1 — Scope resolution

```
product_mode = normalize_mode(mode)
scoped_ids, abort_search = resolve_retrieval_scope(product_mode, document_ids)
if abort_search:
    return insufficient_context_payload()
```

Three cases:

- **Normal mode** — an explicit list of ready document identifiers. Empty means search nothing.
- **Super Focused with a selection** — exactly one identifier, and only if that document is `ready`.
- **Super Focused with no valid selection** — `abort_search` is true, and the function returns the insufficient-context payload **without searching at all**.

That last case is the one to highlight. The tempting shortcut is to fall back to searching everything. But a user who explicitly restricted the scope to one document and then received an answer from a different document has been actively misled. Refusing is correct.

11.3 Step 2 — Turn analysis, without an LLM

`conversation_query.py` analyses the turn deterministically. Its docstring draws the distinction that matters:

This is not a phrase whitelist. Signals are linguistic: anaphora, ordinals, length, content-word overlap, and request type.

It produces:

- **Intent**, one of fourteen: factual, definition, explanation, simplification, elaboration, summary, key_points, comparison, example, listing, why, how, clarification, mixed.
- **Relation**: `new`, `continue`, or `transform`.
- **Subject**: what the question is actually about.
- **needs_rewrite**: whether references remain unresolved.

Doing this without an LLM call is a deliberate performance and reliability choice. It runs in microseconds, it cannot fail due to a provider outage, and it is fully unit-testable. The LLM is reserved for the one thing regex genuinely cannot do.

11.4 Step 3 — Query rewrite, only when needed

```
if history and analysis.needs_rewrite:
    rewritten = rewrite_query(question, history, analysis=analysis)
```

"How does it work?" is unanswerable as a search query — "it" carries no retrievable signal. The rewriter turns it into "How does supervised learning work?" using the conversation.

The guard matters. Rewriting every turn would add latency to every question and risk corrupting queries that were already fine.

11.5 Step 4 — The answer plan

`answer_planner.py` maps intent onto the components a good answer of that type contains. For a definition:

definition	what it is	(required, max 2 sentences)
mechanism	how it works	(max 2 sentences)
types	types or categories	(max 1 sentence)
example	document example	(max 1 sentence)

Soft budget: `BRIEFING_WORD_MIN = 120` , `BRIEFING_WORD_MAX = 240` , `MAX_CITATION_MARKERS = 5` .

The plan can also emit **sub-queries**. Asked "What are the types of X?", the first retrieval may find the definition of X but not the enumeration of types. Up to three sub-queries run supplementary retrievals, and at most **two extra unique chunks** are merged in at the end of the pool, preserving the primary rerank order. This is a bounded, deterministic version of what an agent loop would do with far more latency and far less predictability.

11.6 Steps 5 and 6 — Hybrid retrieval and RRF

```
retrieve_dense() ---+
                    +--> fuse_results() --> dedupe --> RERANK_CANDIDATE_K
retrieve_bm25() ---+
```

Dense side. Embed the query with bge-small, query Chroma for `CANDIDATE_K = 20` candidates, filtered to the allowed document identifiers. The filter is applied server-side as a Chroma `where` clause when there are 32 identifiers or fewer; beyond that the clause is skipped and results are post-filtered, because large `$or` filters become slow and unreliable. Failures retry with `n_results` halved.

BM25 side. Built from the **same Chroma corpus** so the two retrievers never disagree about what exists. Cached with a corpus fingerprint. Two extras:

- **Phrase boost.** Consecutive query tokens appearing together score `PHRASE_BM25_BOOST = 2.5` times higher. "Machine learning" as a phrase beats a document that mentions "machine" and "learning" separately.
- **Page filters.** Front-matter queries restrict to pages 1 through `FRONT_MATTER_MAX_PAGE = 8` .

Fusion. Reciprocal Rank Fusion with `RRF_K = 60` :

```
score(chunk) = sum over retrievers of 1 / (60 + rank_in_that_retriever)
```

A chunk ranked 1 by dense and 1 by BM25 scores $1/61 + 1/61$. A chunk ranked 1 by one and absent from the other scores $1/61$. Agreement is rewarded without ever comparing a cosine distance to a BM25 score — which is the entire point, since those two numbers have no common scale.

11.7 Step 7 — Query-type reserved slots

`query_retrieval.py` recognises question shapes that plain hybrid search handles badly, and reserves rerank slots for hits of the matching type (`QUERY_TYPE_RESERVED_SLOTS = 4` , `QUERY_TYPE_CANDIDATE_BOOST = 10`).

Kind	Trigger	Extra handling
<code>figure_ref</code>	"figure 3", "which figure shows"	Targeted BM25 probe for that figure number
<code>article_ref</code>	"article 5", "clause 12"	Lexical boost on the identifier
<code>front_matter</code>	dedication, preface, foreword	Restrict to pages 1 to 8, phrase boost
<code>why_author</code>	"why does the author"	Author and contrast phrase probes
<code>listing</code>	"types of", "causes of", "how many"	Reserved slots for enumerations

The problem being solved: a figure caption is short, and short passages lose to long prose passages under both embedding similarity and BM25 length normalisation. Without reservation, the correct answer is retrieved but ranked below the noise and falls outside the final pool.

11.8 Step 8 — Cross-encoder reranking

Default model `BAAI/bge-reranker-base` , with `Xenova/ms-marco-MiniLM-L-6-v2` available for A/B comparison via `RERANKER_MODEL` .

The reranker reads the query and the passage **together** rather than embedding them separately, which is much more accurate and much slower — hence running it over roughly twenty candidates instead of the corpus.

Passages are truncated to `RERANK_MAX_CHARS = 800` for scoring only. The truncation is deliberately head-weighted, keeping 75 percent from the beginning and the remainder from the end, because definitions and headings tend to lead. Crucially, **the full chunk text is preserved for the LLM context and for citations** — only the scoring input is shortened.

Raw logits become relevance percentages through a calibrated sigmoid in `rerank_calibration.py` :

```
relevance = sigmoid(raw_score + offset_for_model) * 100
```

`MINILM_LOGIT_OFFSET = 4.0` , `BGE_LOGIT_OFFSET = 0.0` . The MiniLM offset is not a guess; the config comment records the held-out audit that produced it:

Held-out MiniLM calibration (audit): entropy true-positive raw ≈ -4.45 must clear citation/evidence floors; World Cup true-negative raw ≈ -5.98 must not. $\text{sigmoid}(-4.45 + 4.0) \approx 39\%$; $\text{sigmoid}(-5.98 + 4.0) \approx 12\%$.

That is a good thing to be able to point at: a constant chosen by measurement against a known true positive and a known true negative, with the reasoning left in the file.

11.9 Step 9 — Evidence selection

The floors:

Constant	Value	Meaning
EVIDENCE_MIN_RELEVANCE	25	Below this a chunk normally leaves the recall pool
CITATION_MIN_RELEVANCE	30	Below this a chunk cannot be a clickable citation
EVIDENCE_NEAR_DUP_RATIO	0.72	Above this overlap, treat two chunks as duplicates
RERANK_TIE_MARGIN	1.0 logit	Keep a below-floor neighbour this close to the top
EVIDENCE_FALLBACK_MIN_RERANK	-5.0	Floor for the dual-source rescue path

Three gating paths, recorded in the diagnostics as `gating_path` :

- `floor_or_tie` — the normal case: chunks above the floor, plus near-tied neighbours within one logit of the top.
- `dual_source` — the floor emptied the pool, but a chunk was found by *both* dense and BM25. Independent agreement is evidence in itself, so it is rescued.
- `recall_fallback` — keep the top chunks even below the floor, to guarantee the pool is never empty when fusion produced candidates.

That last guarantee is important enough to be an explicit acceptance gate: **a nonempty fused pool must never produce zero slots for the LLM**. The alternative failure is silent and infuriating — the right passage was retrieved, then a threshold discarded it, and the user is told nothing was found.

Also applied here: near-duplicate suppression, so five copies of the same boilerplate do not consume all five slots, and page diversification.

11.10 Step 10 — The recall pool, and the citeable subset

This is the cleanest idea in the retrieval design, and the one most worth explaining carefully.

```
recall pool    = every chunk that reaches the LLM.  Each gets E1, E2, E3...
citation pool  = the subset that is allowed to be a clickable citation
                  (relevance >= CITATION_MIN_RELEVANCE, and allowlisted)
```

From `evidence_state.py` :

Every recall-pool chunk sent to the LLM gets a stable E#. Citeability is a separate UI state.

So a chunk can be in the model's context, be referenced by the model, and still not become a clickable precise citation — instead it appears as `page_only` .

Why separate the two? Because the alternatives are both bad. Give the model only high-confidence chunks and you lose recall: a moderately-scored chunk often contains the one detail that completes an answer. Make every chunk in context fully citeable and you present weak evidence with the same visual authority as strong evidence. Splitting identity from citeability lets the model see everything while the interface still tells the truth about confidence.

The two states are `citeable` and `page_only` , and all three conditions must hold for `citeable` :

```
if citation_eligible and allowlisted and int(relevance) >= CITATION_MIN_RELEVANCE:
    return STATE_CITEABLE
return STATE_PAGE_ONLY
```

11.11 Step 11 — The grounded prompt

`answer_prompt.py` builds an explicit priority hierarchy:

1. DOCUMENT PASSAGES are the only source of document facts, examples, types, quotes, and page-related claims.
2. CONVERSATION is only for resolving references. Never treat a prior assistant message as document evidence.
3. If the passages do not support a claim, do not make that claim.
4. Never invent citations, page numbers, examples, lists, or reasoning.
5. Match the requested style without adding unsupported content.

Rule 2 closes a real and easily-missed hole: without it, the model can cite its own earlier answer as if it were the document, and an error introduced in turn one gets laundered into a "sourced" fact by turn three.

The grounding contract asks the model to classify what it is doing:

- A) Facts a passage states directly - you may report these.
- B) Synthesis that several passages together support - allowed if cautious.
- C) Related material that does not actually answer the question - do not present it as the answer.
- D) Missing information - say so.

Prefer "I couldn't find that in the provided document." over inventing an answer.
Do not fabricate examples, definitions, types, reasons, causes, consequences, comparisons, numbers, page numbers, quotations, or citations.
Never invent a page number.

Category C is the subtle one. A model rarely fabricates from nothing; far more often it presents genuinely retrieved but off-target material as though it answered the question. Naming that failure mode explicitly is more effective than a general instruction to be accurate.

There is also a **relevance-aware caution** injected from the actual scores. If the best passage scored under 50, the prompt says so:

Evidence caution: the best passage is only moderately related. Do not turn a weakly related excerpt into a confident answer.

Passages are formatted as [E1] filename, p. N followed by the full chunk text, and the model is instructed to cite as [E#: "8-15 word verbatim anchor"] , at most five markers.

11.12 Step 12 — Generation and streaming

Non-streaming `/chat` calls `ask_question` with `generate=True` . Streaming `/chat/stream` calls it with `generate=False` , gets back the prompt and the citation payload, and streams the generation itself.

The wire protocol:

```
__CITATIONS__[ ...json... ]__END_CITATIONS__      <- sources, sent first
<answer tokens streaming>
__ANSWER_FINAL__"...__END_ANSWER_FINAL__          <- optional, only if repaired
__CITATIONS_FINAL__[ ...json... ]__END_CITATIONS__ <- verified, enriched sources
```

While tokens stream, `iter_resolved_stream` resolves citation markers incrementally. It has to hold back any trailing text that might turn out to be a partially-arrived marker — if a chunk ends with `[E` , emitting it immediately would flash a broken bracket on screen before the rest arrives.

If generation fails **mid-stream**, the partial answer is deliberately **not saved**. A half-finished answer persisted into the conversation would be indistinguishable later from a complete one.

11.13 Step 13 — Refusal verification

`grounding_verifier.py` handles the inverse failure that most systems ignore:

A blanket "not found" answer is invalid when the recall pool already contains supporting text. Repair by one constrained retry, then by an extractive excerpt. Never invent facts that are not in the passages.

Everyone guards against hallucination. Almost nobody guards against a model refusing when the answer was right there in the context. Both are grounding failures.

The repair ladder:

1. Is this a *blanket* refusal? (not a partial answer that mentions a gap)
2. Does the recall pool actually support the question?
3. If yes: retry once with an anti-refusal addendum appended to the same prompt
4. If it refuses again: build an extractive answer from a real excerpt, capped at 400 characters, with the real E# attached
5. If the pool genuinely does not support it: leave the refusal alone

Step 1 is careful. `is_blanket_refusal` accepts a refusal only when it is short (40 words or fewer) or when removing the refusal phrases leaves 8 tokens or fewer. An answer that says "here are three of the four things you asked about, the fourth is not in the document" is a *good* answer and must not be "repaired".

Note that step 3 reuses the same prompt and the same evidence. Retrieval is never re-run, so the retry cannot quietly change what the answer is grounded in.

11.14 Steps 14 and 15 — Validation and visibility

`finalize_answer_citations` is chapter 12's subject. Then `visible_sources` decides what the interface may display:

1. If the answer contains citation markers -> show exactly those sources
2. If the answer is a refusal -> show nothing
3. Otherwise (grounded answer, no markers) -> show up to 3 overlapping recall chunks as `page_only`

Rule 3 covers the case where the model wrote a correct grounded answer but forgot to emit markers. Rather than show no sources at all, chunks whose token coverage against the answer clears `SUPPORT_MIN_COVERAGE = 0.12` are shown as page-level references, capped at `FALLBACK_MAX_SOURCES = 3`.

Rule 2 is worth stating explicitly in a demo: a refusal shows **zero** sources. Attaching citations to "I could not find that" would imply evidence that by definition does not exist.

11.15 Every failure mode in the pipeline

Failure	Behaviour
Chroma query error	Retry with <code>n_results</code> halved
Reranker will not load	<code>RerankerUnavailableError</code> , fall back to RRF ordering
Fused pool nonempty but selection empty	<code>rrf_fallback_candidates</code> safety net
No chunks at all	"No relevant information found in the document."
All LLM providers fail	Returns the <code>GENERATION_UNAVAILABLE</code> sentinel
Stream fails before the first token	Fall back to the next provider
Stream fails after tokens	<code>StreamInterruptedError</code> ; no fallback, nothing saved
Quota or ownership violation	Rejected before the stream opens, with a real status code

12. Citations, claims, and PDF highlights

The differentiating chapter. If somebody asks what is actually novel here, this is the answer.

12.1 The premise

`claim_validator.py` opens with the assumption that drives every design decision downstream:

The LLM output is untrusted. This module verifies markers against retrieved evidence and enriches citation payloads before they reach the client.

Not "mostly reliable". Untrusted. Which means every claim the model makes about where its information came from is treated as an assertion to be checked, not a fact to be displayed.

12.2 The marker format

<code>[E1]</code>	bare reference
<code>[E1:"uses labeled training examples"]</code>	with a verbatim quote
<code>[E1 quote="..."]</code>	alternative quoted form
<code>[E1, E2]</code>	a group, expanded to <code>[E1][E2]</code>

Quote validation rules from `citation_resolver.py` :

Rule	Value	Why
Maximum length	<code>MAX_QUOTE_CHARS = 120</code>	A citation anchor, not a passage dump
Minimum length	<code>MIN_QUOTE_COMPACT = 8</code> non-space characters	Two words cannot be located uniquely
Forbidden characters	<code>" , [,]</code>	Would break the marker grammar
Forbidden content	<code>x0 , y0 , x1 , y1 , bbox , coord_space</code>	Coordinate payloads are never trusted from the model

That last rule is a genuine adversarial defence with a test to match. If the model emits `[E1: x0=10 y0=20 x1=200 y1=40]` , it is trying to hand the interface geometry directly. Coordinates come from the evidence mapping layer only. Both the backend regex and a frontend test in `frontend/src/lib/citations/adversarial.test.ts` reject them.

12.3 The validation pipeline

`finalize_answer_citations` runs these stages in order:

1. `polish_answer_text` strip prose quote dumps, cap markers at 5
2. `resolve_evidence_markers` drop any E-id that was never assigned
3. `attach_all_quotes_to_sources`
4. `repair_answer_markers` downgrade markers whose quote failed validation
5. `extract_claim_near_marker` recover the sentence each marker is attached to
6. `orchestrate_all_claims` find the passage that really supports each claim
7. `resolve_claim_evidence` map the winner to PDF bounding boxes
8. `annotate + ui_status` derive the presentation state per source
9. `build_evidence_trace` structured observability (optional)

12.4 Stage 2 — Invented identifiers cannot survive

Only E-identifiers that were actually assigned to retrieved passages are kept. If the recall pool produced E1 through E5 and the model writes `[E9]`, that marker is removed.

This closes the most common hallucinated-citation attack: the model invents a plausible reference number and, without validation, the interface renders a citation chip that points at nothing.

12.5 Stage 4 — Quote verification

The check itself is small and is the load-bearing line of the whole feature:

```
def validate_quote_against_chunk(quote: str, chunk_text: str) -> bool:
    cleaned = sanitize_quote(quote)
    if not cleaned:
        return False
    return compact_contains(chunk_text or "", cleaned)
```

`compact_contains` compares after normalising whitespace, because extracted PDF text contains line breaks and irregular spacing that a strict comparison would fail on for text that is genuinely present.

If the quote does not appear in the chunk, the marker is **downgraded to a bare** `[E#]` rather than the whole citation being deleted. This is the right granularity: the passage may well still be the correct source even if the model paraphrased instead of quoting. The unverifiable part is removed; the verifiable part survives.

12.6 Stage 6 — Claim orchestration, the clever part

The insight, from `claim_orchestrator.py`:

For each cited claim, search the full recall pool (not only citeable E# slots), score candidates against the claim with document-agnostic lexical support, and rebind chunk/page when a better supporting passage wins by a clear margin.

The problem it solves: retrieval ranks passages against the **question**. The model then writes several distinct claims. A passage that was the best match for the question as a whole is often not the best support for claim number three specifically. Naive systems cite whatever retrieval ranked first, and the citation lands on the wrong page.

So each claim is re-scored independently across the whole pool:

```
combined score = 0.40 * span support
                + 0.45 * token coverage, IDF-weighted
                + 0.15 * retrieval relevance
```

Coverage is weighted highest deliberately, and the source comment explains why:

Combined ranking: distinctive claim coverage outweighs retrieval rank so a related-but-wrong rank-1 chunk cannot lock the citation.

IDF weighting matters here. If a claim contains a rare term, the passage containing that rare term is almost certainly the source. Common words contribute little. This is what makes the scoring **document-agnostic** — no per-document tuning, no keyword lists.

Rebinding requires clearing two bars, `CLAIM_REBIND_MARGIN = 0.12` and `CLAIM_SUPPORT_MIN = 0.42`:

```
rebind only if:
    winner_score > bound_score + 0.12    (a clear margin, not noise)
    winner_score >= 0.42                 (an absolute support floor)
    and for informative claims, the winner must be classified "supported"
```

The margin exists so that near-ties do not cause the citation to flap between passages on identical questions — the acceptance suite measures exactly this as same-claim instability.

The E-number never changes. Only the chunk, the page, and the highlight region behind it move. From the docstring:

E-ID assignment stays with retrieval. This layer only rebinds the evidence anchor behind an existing E#.

This is what keeps the user experience coherent: `[E2]` in the text stays `[E2]`, while what it points to gets corrected.

12.7 Stage 7 — Localisation and re-anchoring

`claim_localizer.py` grades how well a claim was located, worst to best:

```
unsupported < weak < semantic_span < sentence < exact
```

Confidence thresholds: sentence at 0.42, semantic span at 0.52, and 0.52 required before a highlight is drawn.

If the best support straddles a chunk boundary, `claim_reanchor.py` stitches in the neighbouring chunk (`REANCHOR_NEIGHBOR_RADIUS = 1`, that is plus or minus one). The `prev_chunk_id` and `next_chunk_id` metadata written at index time is what makes this cheap.

An important consequence: **the page follows the highlight, not the retrieval slot**. If re-anchoring finds the supporting sentence on page 8 when the retrieved chunk began on page 7, the citation says page 8 — because that is where the user will actually see the highlighted text.

12.8 The visual content policy

`visual_evidence.py` exists to prevent fake precision:

Native-text quotes keep real span highlights. Visual content is shown as a page (plus caption/snippet) unless a tight, real text box was mapped. Geometry is never invented.

Content types:

<code>native_text</code>	normal text; precise highlights allowed
<code>figure_caption</code>	caption text near a graphic
<code>table</code>	tabular layout
<code>scanned_or_image</code>	no text layer
<code>scanned_ocr</code>	text layer produced by OCR
<code>low_text_layout</code>	too little text to trust the geometry

Two geometric sanity checks catch the failure where a "highlight" is really a box around a whole graphic or a whole table:

- `_MAX_FIGURE_BOX_HEIGHT = 72.0` points. Caption lines are short; a taller box is almost certainly covering the image.
- `_MAX_TABLE_AREA_RATIO = 0.40` of the page. A box covering more than 40 percent of the page is a page reference wearing a highlight's clothing.

Fail either check and the citation is downgraded to a page reference. The adversarial suite tests this directly: scanned content must never present as `precise`, even if an upstream flag claimed a highlight was available.

12.9 What the interface shows

`frontend/src/lib/citations/evidenceStatus.ts` folds all the backend signals into one presentation status:

Status	Meaning	Interface
<code>precise</code>	Real span boxes on the right page	Yellow highlight over the sentence
<code>page_only</code>	Right page, no trustworthy tight box	Page reference plus snippet
<code>passage_only</code>	Supporting text known, no page	Snippet only
<code>unavailable</code>	No location information	Metadata only
<code>invalid_quote</code>	Quote was not in the source	Explicit warning
<code>missing_metadata</code>	Citation was dropped	Not shown as evidence

The inputs it considers are `quoteHighlightAvailable`, `quoteMappingStatus`, `uiStatus`, `contentType`, `evidenceState`, and `pageNumber`. Note the ordering guard: even when `quoteHighlightAvailable` is true, a scanned or low-text content type forces `page_only`. The content-type check overrides the highlight flag, because the flag can be optimistic and the content type cannot.

12.10 Evidence tracing

`evidence_trace.py` records the whole chain per claim: the retrieval chunk, the selected chunk, whether rebinding happened, which candidates were searched, the localisation status, the support status, the UI status, and whether it survived into the final citations.

One design detail worth mentioning because it shows privacy was considered: **passage text is never logged**. The trace records identifiers, statuses, and scores. You can debug why a citation landed where it did without user document content flowing into log files.

12.11 The complete defence stack

Laid out in one place, because "how do you stop hallucination?" deserves a structured answer rather than a single technique:

Layer	Defence
Prompt	Explicit refusal preference; never invent facts, pages, or citations
Scope	Only <code>ready</code> , owned documents; Super Focused aborts rather than widening
Gating	Relevance floors; recall-first fallback guarantees a nonempty pool
Stream	Unknown E-ids and coordinate payloads stripped as tokens arrive
Formatter	Prose quote dumps removed; markers capped at five
Refusal check	A wrong refusal is detected and repaired from real excerpts
Quote check	Quote must exist in the chunk text or it is removed
Claim check	Rebind to the passage that genuinely supports the claim
Geometry	Boxes are never invented; <code>highlight_available</code> requires real spans
Visual policy	Scans, figures, and tables cannot claim sentence precision
Response gate	Generic non-answers are not persisted
Visibility	Refusals show no sources; uncited answers show at most three page refs

Twelve layers, each deterministic, each individually testable. That is the real architectural claim of the project: rather than trusting one large model end-to-end, the pipeline verifies its output in separate deterministic stages.

13. The frontend workspace

13.1 Shape

A single route. `frontend/src/app/page.tsx` is the whole workspace; there are no nested routes and no Next.js API routes, since the FastAPI backend is the API.

```
page.tsx
|
+-- AppLayout ..... shell: sidebar, header, signup modal, toasts
|
+-- Sidebar ..... conversations, document library, upload dialog
+-- WorkspaceHeader .... title, export, new chat, usage, user menu
+-- ChatContainer
|
+-- ScopeBar ..... mode switcher and current scope
+-- MessageList ..... transcript, starter prompts, auto-scroll
|   +-- MessageBubble
|       +-- AnswerMarkdown ..... markdown plus [E#] chips
|       |   +-- InlineCitation
|       +-- SourceList
|           +-- CitationCard
+-- ChatInput ..... composer, send and stop
+-- CitationDrawer ... slide-over PDF panel
    +-- PdfEvidenceViewer .... PDF.js canvas plus overlays
```

Three boot states: a loading skeleton, an empty state with the upload hero when there are no documents, and the chat workspace once a document exists.

13.2 State: four Zustand stores, no Context

`useDocumentStore` — the document list, selection, upload, delete, and status polling. Polling starts at 1.5 seconds, backs off to 4 seconds after 30 seconds, and marks a document failed client-side after a 120-second stall. That last part matters: if the backend dies mid-index, the document would otherwise sit in `embedding` forever with no explanation.

`useChatStore` — conversations, messages, streaming state, the active citation, and the product mode. Persists `docusage_active_conversation` and `docusage_product_mode` to `localStorage`, and holds the `AbortController` that powers stop-generation.

`useAuthStore` — user profile, the usage snapshot, and signup modal state. It registers a global quota handler via `setQuotaHandler`, so a `402` from *any* API call opens the sign-in modal without every call site needing to handle it.

`useToastStore` — transient notifications, plus a `notify` helper usable from non-React code.

Plus one derived hook, `useChatScope`, which centralises the "can the user ask a question right now?" logic so the composer and the scope bar cannot disagree.

The reason for Zustand over Context is worth being able to state: selector-based subscriptions. During streaming, tokens update the store many times per second. With Context, every consumer re-renders on every update. With Zustand, a component that only selects the document list is untouched by chat updates.

13.3 The streaming parser

`frontend/src/lib/api/chat.ts` implements `createStreamParser`, which is more subtle than it first appears because control frames can be split across network reads.

Two mechanisms handle it:

Partial frame holdback. `partialMarkerLength` finds the longest suffix of the buffer that is a prefix of a control marker. If the buffer ends in `__CITATIO`, that text is held rather than emitted, because the rest of the frame is probably in the next read.

Incomplete bracket holdback. Once citations are known, `incompleteBracketLength` holds back a trailing partial `[E`, so a broken bracket never flashes on screen.

Frame handling:

```
__CITATIONS__      first payload; render sources immediately
__ANSWER_FINAL__   supersedes every token streamed so far
__CITATIONS_FINAL__ enriched, verified sources replace the first payload
```

`__ANSWER_FINAL__` sets `answerReplaced`, after which `emitLimit()` returns 0 — tokens that arrived in the same read as the frame are discarded too, because they belong to the superseded draft.

13.4 From a click to a highlight

```
1. User clicks [E2] -> InlineCitation sets activeCitation in the chat store
2. CitationDrawer opens
3. Regions already present from the stream? use them
   Otherwise: GET /documents/{id}/chunks/{chunkId}/evidence?quote=...&claim=...
4. PdfEvidenceViewer loads the PDF through PDF.js with session headers attached
5. resolveEvidenceView() + overlayRectsForPage() convert PDF points to canvas px
6. Yellow overlay divs render with mix-blend-multiply
7. Auto-scroll to the focused highlight
```

Viewer details: lazy page rendering via `IntersectionObserver` with a 1600-pixel prefetch margin, the focus page and its neighbours force-rendered immediately, zoom from 75 to 250 percent, and an "open in new tab" link to `/documents/{id}/file#page=N`.

The overlays use `mix-blend-multiply` so the text stays readable through the highlight, exactly like a physical highlighter pen.

13.5 Never silently dropping a marker

`usedCitationsWithFallback()` creates a stub for any `[E#]` marker present in the answer text but missing from the citation metadata.

The reasoning is a good example of choosing which failure to have. If a marker appears in the text but its metadata went missing, the options are to hide the marker (the user reads a sentence that was supposed to be sourced and sees nothing) or to show a degraded chip. The second is better: visible degradation beats invisible loss. The acceptance gates measure this as **UI dropout**, with a required rate of zero.

13.6 Auth in the client

```
Guest:      generate a UUID -> localStorage 'docusage_guest_session'
              -> X-Guest-Session header on every call
Signed in:  Supabase Google OAuth -> access token
              -> Authorization: Bearer, replacing the guest header
First sign-in: POST /auth/migrate-guest with both credentials
```

If `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY` are unset, the app still runs guest-only and the modal explains that sign-in is unavailable — it does not present a button that cannot work.

13.7 Frontend tests

Five Vitest files, all under `frontend/src/`:

File	Covers
<code>lib/citations/adversarial.test.ts</code>	Coordinate payloads rejected, invented E-ids dropped, scans never precise
<code>lib/citations/markers.test.ts</code>	Marker parsing, streaming bracket holdback, quote attachment
<code>lib/citations/evidenceStatus.test.ts</code>	Figure, table, and scan presentation statuses
<code>lib/api/chat.test.ts</code>	Stream parser, citation frames, final-answer replacement
<code>lib/pdf/coords.test.ts</code>	PDF to canvas coordinate mapping

13.8 The full feature list

Upload by drag-and-drop or browse, with client-side validation. Live processing status with retry. Streaming grounded answers. Two search scopes. Inline numbered citations. An expandable source panel. The PDF evidence drawer with highlights. Conversation history with rename, delete, and search. Export to Markdown. Copy answer, with quote payloads stripped. Regenerate and stop. Guest trial. Google sign-in with trial migration. Proactive quota blocking before a request can fail. Honest evidence badges. A responsive layout with a mobile drawer and a resizable desktop sidebar. Focus traps, ARIA live regions, and a keyboard-operable mode switcher.

14. API reference

Eighteen routes in `backend.py`. Every route except `/health` resolves an identity from either `X-Guest-Session` OR `Authorization: Bearer`.

14.1 Health and account

Method	Path	Purpose
GET	<code>/health</code>	Liveness check. Public. Drives the connection indicator
GET	<code>/me/usage</code>	Usage and limits for the current actor
POST	<code>/auth/migrate-guest</code>	Move finished trial work into the account that just signed in

`/me/usage` returns `tier`, `actor_type`, `pdfs_used`, `pdfs_limit`, `questions_used`, `questions_limit`, `questions_window`, `max_pdf_mb`, and `auth_available`. The frontend uses it for the usage meter and to block actions before they can fail.

14.2 Documents

Method	Path	Purpose
POST	<code>/upload</code>	Upload a PDF; returns <code>{document_id, status}</code> and indexes in the background
GET	<code>/documents</code>	The caller's document library
GET	<code>/documents/{id}</code>	Details and current status. Polled during indexing
GET	<code>/documents/{id}/file</code>	The original PDF, inline, for the viewer
GET	<code>/documents/{id}/chunks/{chunk_id}/evidence</code>	Highlight regions for a quote or claim
GET	<code>/documents/{id}/evidence-summary</code>	Highlight coverage statistics for a document
DELETE	<code>/documents/{id}</code>	Delete, purging Chroma, BM25, evidence, and the file

The file route sets `Cache-Control: private, max-age=60`, `X-Content-Type-Options: nosniff`, and `content_disposition_type="inline"` so the browser renders it rather than downloading it. CORS exposes `Accept-Ranges`, `Content-Range`, and `Content-Length` so PDF.js can issue range requests instead of fetching whole files.

The evidence route takes optional `quote` and `claim` query parameters. With a `claim` it runs `resolve_claim_evidence` (paraphrase-tolerant); with only a `quote` it runs `resolve_quote_evidence` (exact-

ish). A 404 here is normal and simply means no highlight is available.

14.3 Chat

Method	Path	Purpose
POST	/chat	Non-streaming grounded answer with citations
POST	/chat/stream	Streaming answer. What the product uses

Request body:

```
{
  "question": "What is supervised learning?",
  "conversation_id": "abc123",
  "document_ids": ["doc1", "doc2"],
  "mode": "normal",
  "regenerate": false
}
```

Response source objects carry: document_id, filename, page, chunk_id, relevance, evidence_id, snippet, quote, quotes, quote_mapping_status, quote_highlight_available, quote_regions, evidence_state, citation_eligible, ui_status, and content_type.

14.4 Conversations

Method	Path	Purpose
GET	/conversations	List the caller's conversations
POST	/conversations	Create one
GET	/conversations/{id}	Load messages, with stored citations
PATCH	/conversations/{id}	Rename
DELETE	/conversations/{id}	Delete
DELETE	/memory/{conversation_id}	Clear a conversation (legacy path, same effect)

14.5 Status codes

Code	Meaning
200	Success
401	Authentication required or session expired
402	<code>QUOTA_EXCEEDED</code> , with resource, limit, used, and upgrade hint
403	Row belongs to another actor
404	Row genuinely absent, or no evidence available
413	PDF above the size limit

15. Memory, modes, and the LLM router

15.1 Conversation memory

`MEMORY_WINDOW = 6`. `get_history` returns the last six messages of a conversation. Full history is stored in the `messages` table; only the window is sent to the model.

The window exists for three reasons: prompt size (the evidence passages already consume most of the context budget), cost, and quality — distant turns are more likely to distract the model than to help it resolve a reference.

Note that history is used for **reference resolution only**, never as evidence. The prompt states this as a numbered rule, and it is what prevents the model from citing its own earlier answer as if it came from the document.

Regeneration is handled by `_prepare_chat_history`: it deletes the last assistant message, and if the last remaining message is the same user question, drops that too, so the model does not see the question twice.

15.2 The three modes

From `modes.py`:

```
NORMAL      question -> retrieve (optionally document-filtered) -> answer
SUPER_FOCUSED the same pipeline, retrieval scoped to one selected document
AGENTIC      future: plan -> multiple retrieval and tool steps -> grounded answer
              Not implemented. Do not pretend it is autonomous.
```

That last line is in the source. The honesty is deliberate and it is a good detail to volunteer rather than be caught by: `AGENTIC_MODE_ENABLED` defaults to `false`, and `effective_mode()` silently downgrades an agentic request to normal.

The key architectural statement, also from the source:

The RAG pipeline (`ask_question` / `retrieve_candidates`) stays the reusable capability. Modes only change scope and orchestration, not scoring.

`agent_foundation.py` defines the seam a future agent would use — `run_document_qa` as a callable tool named `document_qa` — and its docstring instructs against importing `LangChain` or `LangGraph` there. The reasoning: if an agent loop is added later, it should call the existing verified RAG capability per subtask rather than reimplementing retrieval, because every verification layer in chapter 12 lives inside that capability.

15.3 The LLM router

`llm/router.py`, with the rule in its first line: *the same prompt is sent to each fallback; retrieval is never rerun.*

Default chain:

```
groq -> gemini -> cerebras -> openrouter -> mistral
```

Provider	Default model
Groq	<code>llama-3.1-8b-instant</code>
Gemini	<code>gemini-2.0-flash</code>
Cerebras	<code>llama3.1-8b</code>
OpenRouter	configurable
Mistral	<code>mistral-small-latest</code>
Ollama	local development, <code>qwen2.5:3b</code>

Timeout `LLM_REQUEST_TIMEOUT = 60` seconds, `LLM_MAX_RETRIES_PER_PROVIDER = 1`. Providers that are not configured are skipped rather than attempted and failed.

The streaming fallback rule is the interesting part:

Fallback is allowed only before any token is yielded. After tokens are emitted, a failure terminates the stream.

Once the user is reading text, switching providers would restart the answer from the beginning with different wording — visibly broken. So a mid-stream failure raises `StreamInterruptedError` and the partial answer is not saved.

When every provider fails, the router returns the `GENERATION_UNAVAILABLE` sentinel rather than raising, so the endpoint can surface a clean message.

Logging runs through `llm/sanitize.py`, whose module docstring is a one-line security policy: *messages must never include API keys or auth headers*. Provider errors are redacted before they reach a log or an exception string.

16. How quality is proven

The strongest thing about this project when talking to an engineer: the quality claims are measured and machine-checked, not asserted. This chapter is the evidence.

16.1 The layers of testing

Unit tests	43 Python test files, 5 frontend Vitest files
V1 baseline	45 questions, retrieval and generation metrics
Highlight eval	claim to span to PDF region localisation
Held-out real PDFs	20 adjudicated cases on an unseen document
Adversarial pack	186 tests, must be 100 percent
Acceptance gates	frozen launch bars, fails closed
Launch audit	scenarios, latency, failure injection, human sign-off

Each layer answers a different question. Unit tests ask whether the functions behave. The baseline asks whether retrieval finds the right passages. The highlight eval asks whether citations land in the right place. The adversarial pack asks whether the defences hold under attack. The acceptance gates ask whether this build may ship.

16.2 V1 baseline

`evaluation/run_v1_baseline.py` over a fixed 45-question dataset — 40 answerable, 5 unanswerable — spanning factual, semantic, exact-term, multi-chunk, comparison, conversational, and unanswerable categories.

Latest recorded run:

Metric	Result
Retrieval hit at K	0.900
Page hit at K	0.825
Document hit at K	1.000
Mean reciprocal rank	0.785
Retrieval latency, mean	27.4 ms
Answer keyword hit rate	0.975
Citation page hit rate	0.875
Unanswerable handled correctly	1.000
Refusal rate on unanswerable	1.000
Generation latency, mean	7.85 s

The two numbers to draw attention to are the unanswerable rows. A perfect refusal rate on questions the corpus does not answer is the metric that most directly reflects the project's thesis.

The weakest category is conversational at 0.333 retrieval hit — an honest data point about the limits of query rewriting on follow-up turns, and a better answer to "where does it struggle?" than a vague one.

16.3 Highlight and claim verification

`evaluation/run_highlight_eval.py` , latest run:

Metric	Result
Claim verification accuracy	1.000
Evidence localisation accuracy	1.000
Highlight precision	1.000
Fallback rate	0.222
Unresolved rate	0.000
Quote-region cache hit rate	1.000

Localisation cases 9 of 9; claim cases 4 of 4.

The fallback rate of 22 percent is not a failure — it is the system correctly choosing a page-level reference instead of faking a sentence highlight. Reading it as a defect would be reading the design backwards.

16.4 Held-out real documents

The synthetic fixture is not enough on its own, because a harness that generates its own PDFs can accidentally test its own assumptions. So there is a **held-out operations handbook**

(`evaluation/held_out_handbook.py` , rendered to `evaluation/real_pdfs/held_out_handbook.pdf`) with 20 adjudicated cases: 10 native-text localisation cases and 10 retrieval cases.

Release evaluation indexes that PDF into an **isolated store** under `evaluation/results/held_out/` , so evaluating never writes into the live application corpus.

Two modes:

- **localization** — a gold answer with `[E#]` markers pushed through `finalize_answer_citations` , testing mapping and the citation UI.
- **retrieval** — `ask_question(..., generate=False)` then the gold answer, testing recall-pool hit plus localisation.

16.5 The adversarial pack

`evaluation/run_adversarial_suite.py` . Latest run: **186 tests, 186 passed, 0 failed, 0 errors, 0 skipped**. Exit code 1 on any failure — it is a gate, not a report.

It combines generic adversarial cases with the regression modules:

<code>test_adversarial_suite</code>	<code>test_index_hygiene</code>	<code>test_recall_first_retrieval</code>
<code>test_rerank_calibration</code>	<code>test_query_retrieval</code>	<code>test_claim_orchestrator</code>
<code>test_claim_reanchor</code>	<code>test_grounding_verifier</code>	<code>test_citation_evidence_state</code>
<code>test_citation_resolver</code>	<code>test_claim_validator</code>	<code>test_visual_evidence</code>
<code>test_evidence_trace</code>	<code>test_stream_final_answer</code>	<code>test_real_document_eval</code>

The adversarial cases specifically cover: the same question asked three times for stability, paraphrases, wrong-document scope, invented markers, visual false-precision, and refusals on unrelated questions.

It calls no live LLM and changes no retrieval behaviour, which is what makes it usable as a fast, deterministic gate.

16.6 The acceptance gates

`evaluation/run_acceptance_gates.py` . The docstring states the division of labour:

Eval (item 12) measures. The adversarial pack (item 13) regresses. This module is release policy: hard pass/fail against frozen launch bars.

Gate	Bar
Adversarial pack	100%
Recall at pool (retrieval hit)	at least 92%
Post-rerank context recall	at least 90%
Grounding (supported claims)	at least 90%
Native-text localisation	at least 95%
Wrong page	at most 2%
False-precise highlight	at most 1%
Native-text UI dropout	0%
Rerank zero-result when fused pool nonempty	0%
Same-claim instability, question asked 3 times	at most 5%
Held-out evaluated cases (release)	at least 20, with at least 10 native and 10 retrieval

Latest certificate, from `evaluation/results/acceptance_latest.json`, profile `release`, certified: `true` :

Metric	Result	Bar
Cases evaluated	20	at least 20
Native cases	18	at least 10
Retrieval cases	10	at least 10
Localisation rate	100%	at least 95%
UI dropout rate	0%	0%
Wrong-page rate	0%	at most 2%
False-precise rate	0%	at most 1%
Grounding rate	100%	at least 90%
Retrieval hit rate	100%	at least 92%
Context recall rate	100%	at least 90%

16.7 Why "fails closed" is the important design detail

This is the single best thing in the evaluation setup and the thing to explain if you get one question about it.

Consider the wrong-page gate: at most 2 percent. Now suppose the case catalogue is empty. Zero cases produce zero wrong pages, which is 0 percent, which passes. Every rate gate passes vacuously and the build certifies while having tested nothing.

The module blocks this explicitly:

Empty held-out reports must not certify a ship. Rate gates do not apply until the minimum evaluated-case floor is met (no vacuous 0% wrong-page pass).

Hence the `release_min_evaluated` count gates: at least 20 cases, at least 10 native, at least 10 retrieval. **A count gate must pass before a rate gate is even considered.**

Note also that the current `real_document_latest.json` in the repository shows `case_count: 0` — a run made with an empty catalogue. Under a naive gate design that file would look like a perfect result. Under this design it certifies nothing, which is exactly the intended behaviour and a live illustration of why the count floors exist.

Two profiles: `--profile ci` is the engineering gate (adversarial pack plus the rerank invariant). `--profile release` is the ship stamp and enforces the count floors.

16.8 The launch audit, and the human in the loop

`evaluation/run_launch_audit.py` repeats the held-out suite through the certificate, then runs nine production-audit scenarios, load and latency probes, and failure injection.

Latest run:

Item	Result
Release certified	true
Held-out cases evaluated	20
Production scenarios hit	9 of 9
Same question three times, stable	true
Latency p50	626.6 ms
Latency p95	1104.9 ms (threshold 180,000 ms)
Latency max	1358.7 ms
Engineering passed	true
Launch approved	false
Blockers	<code>human_signoff_required</code>

The four failure-injection probes, all passing:

Probe	What it proves
<code>ghost_document_isolation</code>	Unknown document ids do not fall back to the rest of the corpus
<code>super_focused_empty_aborts</code>	Super Focused with no selection aborts rather than searching the library
<code>empty_allowlist_drops_hits</code>	An empty allowlist drops hits instead of searching all of Chroma
<code>invented_marker_stripped</code>	Invented markers and coordinate payloads never become citations

The row to point at is `engineering_passed: true` with `launch_approved: false`. Every automated check passes and the system still refuses to declare itself ready to ship, because `--signoff` was not given. A human must decide that.

That is a deliberate statement about what automated testing can and cannot establish. Tests prove that the properties you thought to test still hold. They cannot prove you thought of everything.

16.9 Running the suites

```
# Platform layer: identity, quotas, isolation, guest migration
python -m unittest test_platform_quotas test_platform_api

# Retrieval, citation, and evidence
python -m unittest test_claim_validator test_evidence_mapping test_super_focused

# Frontend
cd frontend && npm test

# The evaluation suites
python -m evaluation.run_v1_baseline
python -m evaluation.run_highlight_eval
python -m evaluation.run_real_document_eval
python -m evaluation.run_adversarial_suite
python -m evaluation.run_acceptance_gates --profile ci
python -m evaluation.run_acceptance_gates --profile release
python -m evaluation.run_launch_audit
```

17. Running it and deploying it

17.1 Local development

Requirements: Python 3.12 or newer, Node 18 or newer.

Backend:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
uvicorn backend:app --reload
```

Frontend:

```
cd frontend
npm install
copy .env.example .env.local
npm run dev
```

Open `http://localhost:3000` . Sign-in is optional — with no Supabase credentials the app runs as a guest trial.

The first request is slow. It downloads the embedding and reranker model weights, which takes a few minutes. This is normal and worth saying out loud before a demo so nobody thinks it has hung.

The frontend has a `predev` , `prebuild` , and `postinstall` hook that runs `scripts/copy-pdf-worker.mjs` , copying the PDF.js worker into `public/` . PDF.js needs its worker served from your own origin.

17.2 Configuration

Everything is an environment variable, so the same image runs locally and in production.

Retrieval and generation:

Variable	Default	Purpose
LLM_PRIMARY_PROVIDER	groq	First provider tried
LLM_FALLBACK_PROVIDERS	gemini,cerebras,openrouter,mistral	Fallback order
GROQ_API_KEY	—	Groq credentials
GROQ_MODEL	llama-3.1-8b-instant	Groq model
LLM_REQUEST_TIMEOUT	60	Seconds per request
CHROMA_DB_PATH	./chroma_db	Vector store location
COLLECTION_NAME	ml_notes	Chroma collection
TOP_K	5	Final evidence count
CANDIDATE_K	20	Per-retriever candidate pool
RRF_K	60	Fusion constant
RERANKER_MODEL	bge	bge or minilm, or a full model id
EVIDENCE_MIN_RELEVANCE	25	Recall floor
CITATION_MIN_RELEVANCE	30	Citeable floor
MEMORY_WINDOW	6	Conversation turns sent to the model
AGENTIC_MODE_ENABLED	false	Keep disabled

Note that the shipped `.env.example` sets `TOP_K=3` while `config.py` defaults to `5` when the variable is absent. Not a bug, but worth knowing so you are not surprised by three sources instead of five.

Platform:

Variable	Default	Purpose
DATABASE_URL	sqlite:///./data/documents.db	Relational store
CORS_ORIGINS	http://localhost:3000	Allowed browser origins
AUTH_PROVIDER	none	supabase enables Google sign-in
SUPABASE_URL	—	Supabase project URL
SUPABASE_JWT_SECRET	—	Verifies access tokens
GUEST_TRIAL_ENABLED	true	Allow anonymous use
QUOTA_GUEST_MAX_PDFS	1	Trial document limit
QUOTA_GUEST_MAX_QUESTIONS	15	Trial question limit
QUOTA_USER_MAX_PDFS	5	Signed-in document limit
QUOTA_USER_MAX_QUESTIONS_MONTHLY	100	Signed-in monthly questions
QUOTA_MAX_PDF_MB	25	Upload size limit

Frontend:

Variable	Purpose
NEXT_PUBLIC_API_URL	Backend base URL
NEXT_PUBLIC_SUPABASE_URL	Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase anonymous key
NEXT_PUBLIC_USE MOCK	Return stub data instead of calling the API

17.3 Deployment

Frontend on Vercel. Import the repository, set the root directory to `frontend`, and set the three `NEXT_PUBLIC_` variables.

API in Docker on one VM. The compose file is written for a free-tier virtual machine (its comment names an Oracle 2 OCPU / 12 GB instance):

```
cp .env.example .env      # fill in GROQ_API_KEY, CORS_ORIGINS, Supabase secret
docker compose up -d --build
curl http://localhost:8000/health
```

Three stateful paths are mounted so a rebuild or restart never loses data:


```
volumes:
  - ./data:/app/data          # PDFs and the SQLite database
  - ./chroma_db:/app/chroma_db # embeddings
  - model_cache:/root/.cache  # model weights
```

The health check has a `start_period` of 300 seconds, because first boot downloads the models and would otherwise be marked unhealthy and restarted in a loop.

Supabase. Create a project, enable only the Google provider, add the Vercel URL as a redirect URL, then set `AUTH_PROVIDER=supabase` and `SUPABASE_JWT_SECRET` on the API.

17.4 Migration paths already accounted for

Change	What it touches
SQLite to Postgres	<code>DATABASE_URL</code> , plus extending <code>database/connection.py</code> . Migrations and product code unchanged
Different identity provider	Replace <code>app_platform/auth/supabase_jwt.py</code> only
Different reranker	Set <code>RERANKER_MODEL</code> ; aliases <code>bge</code> and <code>minilm</code> are built in
Different LLM provider	Reorder <code>LLM_PRIMARY_PROVIDER</code> and <code>LLM_FALLBACK_PROVIDERS</code>
Retune limits	Edit <code>.env</code> and restart; limits are read per request

18. Limits and honest answers

Volunteer these rather than being caught by them. Knowing your system's boundaries reads as competence; being surprised by them does not.

18.1 Scanned PDFs

A PDF with no text layer cannot be indexed at all — extraction yields nothing and the document ends as `failed` with a reason. A PDF whose text layer came from OCR can be indexed, but is classified as `scanned_ocr` and its citations are downgraded to page references, because OCR span geometry is not reliable enough to draw a sentence box.

There is no OCR in the pipeline. Adding it (Tesseract, or a cloud OCR service) is the most obvious next feature.

18.2 Agentic mode is not built

It is a documented seam, a disabled flag, and a tool-name constant. Nothing more. Say so plainly — the source says so too.

18.3 SQLite

Fine for a single VM with modest concurrency. It is a single writer, so heavy concurrent write load would be a problem. The abstraction for Postgres is in place; the migration has not been done because nothing yet requires it.

18.4 Conversational follow-ups are the weakest category

The V1 baseline measured 0.333 retrieval hit on conversational questions against 0.9 to 1.0 elsewhere. Query rewriting helps but does not solve it. This is the honest answer to "where does it struggle?"

18.5 The cold-start delay

The first request after a fresh deployment downloads embedding and reranker weights. Minutes, not seconds. Mitigated by the Docker model-cache volume and the 300-second health-check grace period, but present.

18.6 English only

The embedding model is `bge-small-en-v1.5` — the `en` is the constraint. The BM25 tokeniser is also alphanumeric, which does not suit languages that do not word-break on spaces.

18.7 One page of the corpus at a time

Retrieval returns five chunks by default. A question requiring synthesis across thirty pages will not be answered well. The answer planner's sub-queries widen this a little, bounded at two extra chunks. This is a genuine architectural limit, not a tuning issue.

18.8 The leftover Streamlit prototype

`app.py` and `components/` are not the product and are not deployed. Kept for history.

18.9 The launch audit is not signed off

`launch_approved: false`, blocker `human_signoff_required`. Every automated check passes. The human gate is deliberately unsatisfied, which is the correct state for a beta.

18.10 What would come next

Ranked by value, which is a better answer than an unordered wish list:

1. **OCR for scanned documents.** The largest category of PDF the system currently cannot serve.
 2. **Better conversational retrieval.** The measured weak spot.
 3. **Postgres.** Needed before real concurrency.
 4. **Table-aware extraction.** Tables currently degrade to page references.
 5. **Cross-document synthesis.** Answering across many documents at once rather than retrieving from a shared pool.
 6. **Actually building agentic mode.** The seam exists; the loop does not.
-

19. File-by-file glossary

Grouped by role. Use this when someone asks "where is that in the code?"

API and orchestration

File	Purpose
<code>backend.py</code>	FastAPI routes, streaming protocol, request wiring
<code>rag.py</code>	Retrieval and generation orchestration; <code>ask_question</code>
<code>config.py</code>	Every retrieval and generation tuning value
<code>modes.py</code>	Product modes: normal, super focused, agentic
<code>agent_foundation.py</code>	Agentic seam. Foundation only, not autonomous

Ingestion

File	Purpose
<code>pdf_extraction.py</code>	PyMuPDF primary, pypdf fallback
<code>chunking.py</code>	Structure-aware, document-level chunking
<code>indexer.py</code>	The indexing job and its status transitions
<code>evidence_mapping.py</code>	Chunk text to page slices to PDF bounding boxes
<code>index_hygiene.py</code>	Corpus isolation and three-store consistency
<code>document_paths.py</code>	Resolve on-disk PDF paths

Retrieval

File	Purpose
<code>hybrid_retrieval.py</code>	Dense plus BM25, RRF, rerank orchestration
<code>bm25_index.py</code>	BM25 over the same Chroma corpus
<code>reranker.py</code>	Cross-encoder scoring and evidence selection
<code>rerank_calibration.py</code>	Score calibration and pre-LLM diagnostics
<code>query_retrieval.py</code>	Query-type retrieval profiles
<code>retrieval_logger.py</code>	Retrieval debug logging

Query understanding and prompting

File	Purpose
<code>conversation_query.py</code>	Deterministic turn analysis
<code>query_rewriter.py</code>	LLM rewrite into a standalone query
<code>answer_planner.py</code>	Intent to briefing components and sub-queries
<code>answer_prompt.py</code>	The grounding contract
<code>evidence_focus.py</code>	Passage roles and citation allowlist
<code>answer_formatter.py</code>	Removes quote dumps, caps markers

Citation and evidence validation

File	Purpose
<code>citation_resolver.py</code>	Marker parsing and validation, including mid-stream
<code>claim_validator.py</code>	The finalisation pipeline
<code>claim_localizer.py</code>	Claim to span to region localisation
<code>claim_orchestrator.py</code>	Claim-centric rebinding
<code>claim_reanchor.py</code>	Cross-chunk and cross-page re-anchoring
<code>quote_evidence.py</code>	Query-time quote to PDF mapping, cached
<code>visual_evidence.py</code>	Figure, table, and scan policy
<code>grounding_verifier.py</code>	Grounding verification and anti-refusal
<code>evidence_state.py</code>	Citeable versus page-only; visible sources
<code>evidence_trace.py</code>	Production observability
<code>response_validator.py</code>	Gate before persistence

Generation

File	Purpose
<code>llm_service.py</code>	Stable generation facade
<code>llm/router.py</code>	Provider chain and fallback rules
<code>llm/base.py</code>	Provider interface and attempt records
<code>llm/errors.py</code>	Error taxonomy; never leaks keys
<code>llm/sanitize.py</code>	Secret redaction for logs
<code>llm/http_util.py</code>	Shared HTTP helpers
<code>llm/providers/</code>	Groq, Gemini, Cerebras, OpenRouter, Mistral, Ollama

Platform

File	Purpose
<code>app_platform/settings.py</code>	Env-driven deployment settings
<code>app_platform/auth/context.py</code>	<code>RequestContext</code> : who is making this request
<code>app_platform/auth/dependency.py</code>	FastAPI identity dependency
<code>app_platform/auth/guest.py</code>	Anonymous trial sessions
<code>app_platform/auth/supabase_jwt.py</code>	HS256 token verification
<code>app_platform/quotas/limits.py</code>	Per-tier limits from env
<code>app_platform/quotas/service.py</code>	Quota checks and usage recording
<code>app_platform/guards/ownership.py</code>	Row-level access control
<code>app_platform/errors.py</code>	Stable API error shapes

Data

File	Purpose
<code>database/db.py</code>	Schema creation and migrations
<code>database/connection.py</code>	Decides where the database lives
<code>database/document_store.py</code>	Document rows and ownership
<code>database/document_service.py</code>	Document listing and removal
<code>database/evidence_store.py</code>	Chunk evidence sidecar
<code>database/user_store.py</code>	Signed-in profiles
<code>database/guest_store.py</code>	Guest sessions and counters
<code>database/usage_store.py</code>	Period-scoped question counters
<code>memory/manager.py</code>	The conversation window
<code>memory/store.py</code>	Conversation and message persistence
<code>memory/formatter.py</code>	History formatting

Frontend

File	Purpose
<code>frontend/src/app/page.tsx</code>	The workspace
<code>frontend/src/store/useDocumentStore.ts</code>	Documents and status polling
<code>frontend/src/store/useChatStore.ts</code>	Conversations, streaming, active citation
<code>frontend/src/store/useAuthStore.ts</code>	User, usage, signup modal
<code>frontend/src/lib/api/chat.ts</code>	Stream parser and citation mapping
<code>frontend/src/lib/api/client.ts</code>	Session headers and typed errors
<code>frontend/src/lib/citations/markers.ts</code>	Marker parsing for rendering
<code>frontend/src/lib/citations/evidenceStatus.ts</code>	Presentation status derivation
<code>frontend/src/lib/pdf/coords.ts</code>	PDF points to canvas pixels
<code>frontend/src/lib/pdf/pdfjs.ts</code>	PDF.js loader and worker wiring
<code>frontend/src/components/chat/PdfEvidenceViewer.tsx</code>	Canvas rendering and overlays
<code>frontend/src/components/chat/CitationDrawer.tsx</code>	The slide-over evidence panel

Evaluation

File	Purpose
<code>evaluation/run_v1_baseline.py</code>	The 45-question baseline
<code>evaluation/run_highlight_eval.py</code>	Highlight and claim verification
<code>evaluation/run_real_document_eval.py</code>	Held-out real PDFs
<code>evaluation/run_adversarial_suite.py</code>	The adversarial and regression gate
<code>evaluation/run_acceptance_gates.py</code>	Release certification
<code>evaluation/run_launch_audit.py</code>	Launch readiness audit
<code>evaluation/held_out_handbook.py</code>	Generates the held-out PDF
<code>evaluation/held_out_runtime.py</code>	Isolated index for held-out runs
<code>evaluation/acceptance_gates.py</code>	Threshold definitions and gate logic

20. Question and answer appendix

Eighty-eight questions, grouped. Find the group, find the question.

A. Product and demo

A1. What is DocuSage in one sentence? A web app where you upload a PDF, ask questions, and every important claim in the answer carries a citation you can click — which opens the PDF and highlights the exact sentence the claim came from.

A2. Who is it for? Students and anyone who has to read long documents and be sure of what they found. It is currently free for students with no paid tier.

A3. Why not just paste the PDF into ChatGPT? Three reasons. Long documents exceed the context window, so something must decide what to send. Even when it fits, you get no verifiable pointer back to the source — you have to trust the answer. And a general chatbot will happily answer from world knowledge when the document is silent, which is precisely the failure mode this project exists to prevent.

A4. What is the single most important feature? Click-to-source highlighting, backed by quote verification. The demo moment is clicking a citation and watching the exact supporting sentence get highlighted in the original PDF.

A5. What happens when the document does not contain the answer? You get an explicit statement that the information was not found, and **no sources at all**. Attaching citations to a refusal would imply evidence that does not exist.

A6. Why is refusing a feature? Because the value of the tool is that when it *does* answer, you can trust it. That guarantee requires the willingness to say no. A system that always answers gives you no signal about which answers to trust.

A7. Do I need to sign up? No. You can upload and ask immediately. The trial covers one PDF and fifteen questions. Signing in with Google — also free — raises it to five PDFs and one hundred questions a month.

A8. What happens to my trial work if I sign in? It moves with you. `POST /auth/migrate-guest` reassigns your documents and conversations to the new account.

A9. Can I keep signing in to reset the free questions? No. The migration replays your trial question count against the new monthly allowance, and a trial can only be claimed once.

A10. What file types are supported? PDF only. And it must have a real text layer — a pure scan cannot be indexed.

A11. What is Super Focused mode? Restricting retrieval to a single selected document. Useful when you have several similar files and want certainty about which one the answer came from. It is enforced in the backend, not merely filtered in the interface.

A12. Can I export a conversation? Yes, to Markdown, from the workspace header.

A13. Can I stop a long answer? Yes. The partial answer is kept and marked as stopped rather than discarded.

A14. Why do answers look like briefings instead of essays? The answer planner maps intent to components — overview, key points, mechanism, example — with a soft budget of 120 to 240 words and at most five citation markers. A structured briefing is easier to verify claim by claim than a wall of prose.

A15. What is the best five-minute demo order? Frame the problem before opening the app; upload; ask a question the document answers; click a citation for the highlight; then ask something the document does not cover to show the refusal. Chapter 4 has the script.

B. Architecture

B1. Describe the architecture in thirty seconds. A Next.js frontend calls a FastAPI backend. The backend resolves identity, quotas, and ownership first, then passes a filtered list of document identifiers into an unchanged RAG core. The core runs hybrid retrieval over ChromaDB and BM25, reranks with a cross-encoder, generates with Groq, then validates every claim and citation before responding. ChromaDB holds embeddings, SQLite holds metadata and evidence geometry, and PDFs sit on disk.

B2. Why is authentication separated from RAG? So retrieval never has to remember to check. The guard filters document identifiers upstream; the RAG core has no isolation logic at all, which means there is no code path where a missing check leaks data. It also keeps retrieval benchmarkable without constructing users and tokens.

B3. How do you prevent one user's documents from reaching another user?

`visible_document_ids` filters the identifier list, applied twice — once to what the client requested and again to whatever the scope resolved. Combined with `resolve_retrieval_scope`, retrieval always receives an explicit allowlist. An empty list means search nothing; it never means search everything.

B4. What if I guess someone else's document UUID? `403`, with the message "Document not found." The status code differs from a real 404 for the server's own diagnostics, but the message does not confirm that the identifier exists.

B5. Why SQLite and ChromaDB rather than one database? They answer different questions. Chroma does vector similarity; SQLite does relational metadata, ownership, and the evidence geometry sidecar. Postgres with pgvector could do both, and the abstraction is in place for it, but it would add an external service to a single-VM deployment for no current benefit.

B6. How would you move to Postgres? Change `DATABASE_URL` and extend `database/connection.py`. The migrations and product code are unchanged. The connection layer was written specifically to make this a one-file change.

B7. Why is the RAG core described as "unchanged"? Because the platform layer was added around it without modifying it. Auth, quotas, and ownership were introduced entirely in the API shell. That is the proof the seam is real rather than aspirational.

B8. Why does retrieval run once when the endpoint streams? `ask_question(generate=False)` does the retrieval and builds the prompt, then the endpoint streams generation itself. The obvious bug would be retrieving twice — once to build the prompt and once to generate — which wastes compute and risks two different evidence sets.

B9. Why send citations before the answer text? So the interface can render the source list immediately while tokens stream in. A second, enriched frame arrives after validation.

B10. Why check quotas before opening the stream? Once a streaming response starts, the HTTP status is already 200. Checking first means a blocked request gets a real `402` or `403` the client can act on, rather than an error hidden inside a successful-looking body.

B11. Is this a microservice architecture? No, and deliberately not. One API process, one frontend. At this scale microservices would add operational complexity and network hops for no benefit. The internal seams — platform layer, RAG core, LLM router — are where services would split if that ever became necessary.

B12. What is the deployment topology? Frontend on Vercel. API in Docker on a single small VM, with `./data`, `./chroma_db`, and the model cache mounted so restarts do not lose data.

C. Ingestion and indexing

C1. Walk me through what happens to an uploaded PDF. Quota and size checks, a database row with status `uploaded`, the file written to disk, then a background task: extract with PyMuPDF, chunk at 1200 characters with 150 overlap, map every chunk to PDF bounding boxes, embed in batches of 50, write to Chroma, invalidate the BM25 cache, and mark it `ready`.

C2. Why does upload return before indexing finishes? Indexing takes tens of seconds. Holding an HTTP request open that long invites proxy timeouts. The client polls `GET /documents/{id}` instead, which is what drives the status checklist.

C3. Why two PDF extraction engines? They fail differently. PyMuPDF is faster and provides span geometry, which highlights require. Some PDFs yield almost nothing from it but read fine with pypdf. The fallback triggers below 50 total stripped characters and costs nothing when the primary works.

C4. Why 1200 characters per chunk? It sits comfortably inside the roughly 512-token window of `bge-small-en-v1.5` while giving more context than the earlier 800-character page-isolated setup.

C5. Why 150 characters of overlap? So a sentence straddling a boundary survives intact in at least one chunk. Without overlap, a definition split across the boundary retrieves poorly from both halves.

C6. Why chunk the whole document rather than page by page? Because content does not respect page breaks. A definition beginning at the bottom of page 7 and finishing on page 8 would be split by page-isolated chunking and neither half would retrieve well. Page boundaries are kept as metadata instead of enforced as splits.

C7. How are headings detected? Deterministically, with three conservative regex patterns: `UNIT I` style markers, numbered sections like `1.2` or `2.5.1`, and short all-capitals lines. No LLM call. Each chunk inherits `section_title` and a slugified `section_id`.

C8. What is the chunk identifier format? `{document_id}_{index}`. Deterministic, so re-indexing produces the same identifiers, and neighbour links for cross-chunk re-anchoring are trivial.

C9. When are highlight coordinates computed? At **index time**, not question time. Chunk text is aligned to page spans and the bounding boxes are stored in `chunk_evidence`. Query-time work is only narrowing to a specific quote, and that is cached.

C10. How is chunk text aligned to page coordinates? A four-level cascade: exact substring, whitespace-normalised compact comparison with an index map, compact substring search, then hyphen-stripped compact search. Each level is more forgiving, because extracted PDF text contains non-breaking spaces, typographic quotes, ligatures, and line-break hyphens.

C11. What if a chunk spans a page break? Pages are joined with `\n\n`, so such a chunk is not a substring of any single page. It is split on the join and each part mapped independently. The `join_recovered` column records that this happened.

C12. Can the system invent a highlight box? No. From the source: *never invent boxes*:
`highlight_available` is true only when every non-empty slice mapped to at least one real span bbox.

C13. Why embed in batches of 50? Memory. On a 12 GB VM, embedding a large document in one pass grows memory until the process is killed. Batching with explicit garbage collection keeps it flat.

C14. What happens if evidence mapping fails? It is non-fatal. The document still indexes and answers; citations degrade to page level. A working document with page citations beats a document that refused to index because its geometry was unusual.

C15. What if a PDF has no extractable text? The index is purged, status becomes `failed`, and `index_error` is set to `INDEX_FAILURE_NO_TEXT`. The interface offers a retry rather than leaving a question-answerable document that has no content.

C16. How is re-indexing made safe? `purge_chroma_document` removes existing vectors first, so you never end up with two generations of chunks for one document producing duplicate citations.

D. Retrieval

D1. Why hybrid retrieval instead of just embeddings? Embeddings are good at meaning and bad at exact strings. Ask for "Article 5" or "Figure 3" and a dense vector returns something semantically adjacent. BM25 is the opposite — precise about tokens, blind to meaning. Running both covers both failure modes.

D2. What is Reciprocal Rank Fusion? A method for merging ranked lists using only ranks: `score = sum of 1/(60 + rank)` across retrievers. A chunk ranked first by both scores twice as high as one ranked first by only one.

D3. Why RRF instead of normalising and adding the scores? Because a cosine distance and a BM25 score have no common scale. Normalising them against each other requires tuning that breaks whenever the corpus changes. RRF uses only ordering, so the problem disappears.

D4. Why 60 for the RRF constant? It is the standard value from the original RRF paper, and it is configurable via `RRF_K`. It damps the difference between adjacent top ranks so a single retriever cannot dominate on a narrow margin.

D5. What is a cross-encoder reranker and why use one? It reads the query and the passage together rather than embedding them separately, which is far more accurate and far too slow for a whole corpus. So it runs over about twenty fused candidates. This is where most of the retrieval quality comes from.

D6. Why truncate passages to 800 characters for reranking? The cross-encoder has roughly a 512-token budget shared between query and passage. Truncation is head-weighted (75 percent from the start) because definitions and headings lead. The **full text is still what the LLM sees** — only the scoring input is shortened.

D7. What happens if the reranker fails to load? `RerankerUnavailableError`, and the system falls back to RRF ordering. Quality drops; the product keeps working.

D8. What do the relevance floors mean? `EVIDENCE_MIN_RELEVANCE = 25` is the floor for staying in the recall pool. `CITATION_MIN_RELEVANCE = 30` is the stricter floor for being a clickable citation. Between them, a chunk can be in the model's context but presented only as a page reference.

D9. How is a raw reranker logit turned into a percentage? `sigmoid(raw_score + model_offset) * 100`. The offset is 0 for BGE and 4.0 for MiniLM.

D10. Where did the MiniLM offset of 4.0 come from? A held-out audit, recorded in the config comment: a known true positive scored about -4.45 raw and had to clear the floors; a known true negative scored about -5.98 and had to fail them. With the offset, they calibrate to roughly 39 percent and 12 percent — either side of the 25 and 30 floors.

D11. What if the floors filter out everything? Three gating paths prevent an empty pool: `floor_or_tie` for the normal case, `dual_source` to rescue a chunk found independently by both retrievers, and `recall_fallback` to keep the top chunks regardless. A nonempty fused pool producing zero LLM slots is an acceptance-gate failure with a required rate of zero.

D12. Why is an empty pool such a serious failure? Because it is silent and infuriating. The right passage was retrieved, then a threshold discarded it, and the user is told nothing was found. That is worse than a weak answer, because it destroys trust in the search rather than in one answer.

D13. What is the difference between the recall pool and the citation pool? The recall pool is every chunk the model sees; each gets a stable E-number. The citation pool is the subset allowed to become a clickable citation. Everything else appears as `page_only`.

D14. Why split them? Both alternatives are worse. Send only high-confidence chunks and you lose recall — a moderately-scored chunk often holds the detail that completes the answer. Make everything fully citeable and weak evidence gets the same visual authority as strong evidence. Splitting identity from citeability lets the model see everything while the interface still tells the truth.

D15. What are query-type reserved slots? Figure references, article references, front matter, author-intent questions, and listings each get extra targeted BM25 probes and up to four reserved rerank slots. Without this, short passages such as figure captions get retrieved but ranked below long prose and fall out of the final pool.

D16. Why do front-matter queries restrict to pages 1 to 8? Because dedications, prefaces, and forewords are structurally at the front, and the words in them recur throughout the body. The page filter uses document structure that pure text search cannot see.

D17. What are answer-plan sub-queries? When the intent implies multiple facets — "what are the types of X?" — up to three supplementary retrievals run, contributing at most two extra unique chunks at the end of the pool. A bounded, deterministic alternative to an agent loop.

D18. How does retrieval know which documents it may search? `live_searchable_document_ids()` returns SQLite rows with status `ready`, then ownership filtering narrows that. SQLite is the authority, not Chroma — which is why a deleted document cannot resurface even if a vector were somehow orphaned.

D19. Why filter Chroma server-side only up to 32 identifiers? Large `$or` filters become slow and unreliable. Beyond 32 the clause is skipped and results are post-filtered instead.

D20. What is near-duplicate suppression? Chunks overlapping above `EVIDENCE_NEAR_DUP_RATIO = 0.72` are treated as duplicates so repeated boilerplate cannot consume every evidence slot.

D21. What is the tie margin? `RERANK_TIE_MARGIN = 1.0` logit. A below-floor chunk within one logit of the top chunk is kept, because near-ties often mean both passages are relevant. It is tight enough that a genuine true negative does not sneak through.

D22. Why is `SIMILARITY_THRESHOLD` still in the config? It is a legacy dense-distance gate from Phase 1 and 2, and the code comments state explicitly that it is **not** applied to reranker scores. Applying a distance threshold to a cross-encoder logit would be a category error.

E. Generation and grounding

E1. Which model writes the answers? Groq's `llama-3.1-8b-instant` by default, with Gemini, Cerebras, OpenRouter, and Mistral as fallbacks. Ollama is supported for local development.

E2. Why Groq? Very fast inference with a free tier, which matters for a streaming interface where perceived latency is what the user notices.

E3. What if the provider is down? The router tries the next one in the chain with the same prompt. Retrieval is never re-run. If all fail, it returns the `GENERATION_UNAVAILABLE` sentinel.

E4. Can it switch providers mid-answer? No. Fallback is allowed only before the first token. Switching after the user started reading would restart the answer with different wording. A mid-stream failure raises `StreamInterruptedError` and the partial answer is not saved.

E5. What stops the model using its own world knowledge? The prompt's priority hierarchy states that document passages are the only source of document facts, plus an explicit list of things never to fabricate — examples, definitions, types, reasons, causes, comparisons, numbers, page numbers, quotations, citations. And then the validation layer removes anything unverifiable regardless of what the prompt achieved.

E6. Why does the prompt say conversation history is not evidence? Otherwise the model can cite its own earlier answer as though it were the document. An error introduced in turn one would be laundered into a "sourced" fact by turn three.

E7. What is the A/B/C/D grounding contract? Four categories the model must distinguish: (A) facts stated directly, reportable; (B) synthesis across passages, allowed if cautious; (C) related material that does not answer the question, must not be presented as the answer; (D) missing information, say so.

E8. Why is category C called out separately? Because it is the most common real failure. Models rarely fabricate from nothing; far more often they present genuinely retrieved but off-target material as if it answered the question. Naming that failure explicitly works better than a general instruction to be accurate.

E9. Does the prompt know how confident retrieval was? Yes. If the best passage scored under 50, the prompt says the best passage is only moderately related and warns against turning a weakly related excerpt into a confident answer.

E10. What is the anti-refusal verifier? `verify_and_repair_refusal`. If the model refused but the recall pool does contain support, it retries once with an addendum on the same prompt; if it refuses again, it builds an extractive answer from a real excerpt capped at 400 characters with the real E-number attached.

E11. Why guard against wrong refusals at all? Because that is a grounding failure too. Everyone guards against hallucination; almost nobody guards against a model refusing when the answer was in its context. Both damage trust.

E12. Could that "repair" invent an answer? No. The retry reuses the same prompt and the same evidence, and the extractive fallback quotes real text from a real passage. It cannot introduce content that was not retrieved.

E13. How do you avoid repairing a legitimate partial answer? `is_blanket_refusal` accepts a refusal only when it is 40 words or fewer, or when removing the refusal phrases leaves 8 tokens or fewer. An answer that says "three of the four things you asked are here, the fourth is not" is a good answer and is left alone.

E14. What if the model streamed something wrong that got repaired? The server sends `__ANSWER_FINAL__` with the corrected answer, and the client discards every token already displayed — including tokens that arrived in the same network read as the frame.

E15. What are the conversation intents? Fourteen: factual, definition, explanation, simplification, elaboration, summary, key_points, comparison, example, listing, why, how, clarification, mixed. Plus a relation of new, continue, or transform.

E16. Why is turn analysis deterministic rather than an LLM call? Speed (microseconds), reliability (no provider dependency), and testability. The LLM is reserved for the one thing regex cannot do — resolving a reference into a standalone query.

E17. When is the query rewritten? Only when history exists and analysis says references are unresolved. "How does it work?" carries no retrievable signal; the rewriter turns it into "How does supervised learning work?" Rewriting every turn would add latency and risk corrupting queries that were already fine.

F. Citations, claims, and highlights

F1. What does [E1] mean? Evidence 1 — the first chunk in the recall pool. The interface renders it as a superscript ¹. The numbers are assigned by retrieval order and are stable throughout the answer.

F2. Can the model invent a quote? It can emit one, and it will be caught. `validate_quote_against_chunk` checks the quote against the retrieved chunk text with whitespace-normalised comparison. If it is not there, the quote is removed and the marker downgraded to a bare `[E#]`.

F3. Why downgrade rather than delete the whole citation? Because the passage may still be the correct source even though the model paraphrased instead of quoting. Removing the unverifiable part while keeping the verifiable part is the right granularity.

F4. Can the model invent an evidence number? It can emit `[E9]` when only E1 to E5 exist, and `resolve_evidence_markers` removes it. Only identifiers actually assigned to retrieved passages survive.

F5. What if the model emits coordinates? Rejected. `[E1: x0=10 y0=20 ...]` is caught by a dedicated regex, and a frontend adversarial test covers it independently. Geometry comes only from the evidence mapping layer.

F6. What are the quote length limits and why? Maximum 120 characters, minimum 8 non-space characters. The maximum keeps a citation an anchor rather than a passage dump; the minimum exists because two short words cannot be located uniquely on a page.

F7. What is claim orchestration? Retrieval ranks passages against the **question**, but the model writes several distinct claims. For each claim, the orchestrator re-scores the whole recall pool and rebinds the citation to the passage that actually supports that specific claim.

F8. How is claim support scored? $0.40 * \text{span support} + 0.45 * \text{IDF-weighted token coverage} + 0.15 * \text{retrieval relevance}$.

F9. Why is coverage weighted above retrieval rank? So that a related-but-wrong rank-1 chunk cannot lock the citation. The source comment says exactly this. Retrieval rank reflects the question; coverage reflects the claim.

F10. Why IDF weighting? Because a rare term in a claim is strong evidence about which passage it came from, while common words say almost nothing. It also makes the scoring document-agnostic — no per-document tuning or keyword lists.

F11. When does rebinding happen? The winner must beat the currently bound chunk by `CLAIM_REBIND_MARGIN = 0.12` and clear an absolute floor of `CLAIM_SUPPORT_MIN = 0.42`. For informative claims it must also classify as "supported".

F12. Why require a margin instead of just taking the best score? So near-ties do not cause the citation to flap between passages on identical questions. The acceptance suite measures exactly this as same-claim instability, capped at 5 percent.

F13. Does the E-number change when rebinding happens? No. Only the chunk, page, and highlight region behind it move. `[E2]` stays `[E2]` in the text while what it points to gets corrected.

F14. What is re-anchoring? If the supporting text spans a chunk boundary, the neighbouring chunk (plus or minus one) is stitched in using the `prev_chunk_id` and `next_chunk_id` metadata written at index time.

F15. Which page does a citation report? The page of the **highlight**, not the page where the retrieved chunk started. If re-anchoring finds the sentence on page 8 while the chunk began on page 7, the citation says 8 — because that is where the user will see the highlighted text.

F16. What are the localisation statuses? Worst to best: `unsupported`, `weak`, `semantic_span`, `sentence`, `exact`. Sentence requires 0.42 confidence, semantic span 0.52, and 0.52 is required before a highlight is drawn.

F17. Why do some citations show only a page? Several reasons: scanned or OCR content, a figure caption or table, a mapped box that failed the geometric sanity checks, or a relevance below the citation floor. The interface says which rather than pretending.

F18. What are the geometric sanity checks? A figure box taller than 72 points is almost certainly covering the graphic rather than the caption. A table box covering more than 40 percent of the page is a page reference in disguise. Either check failing downgrades to a page reference.

F19. What are the presentation statuses? `precise`, `page_only`, `passage_only`, `unavailable`, `invalid_quote`, `missing_metadata`.

F20. Can a scanned page ever show a precise highlight? No. Even if `quoteHighlightAvailable` is true, a scanned or low-text content type forces `page_only`. The content-type check overrides the highlight flag, because the flag can be optimistic and the content type cannot. There is an adversarial test for exactly this.

F21. What happens to a marker whose metadata went missing? `usedCitationsWithFallback` creates a visible stub. Visible degradation beats invisible loss — if the marker vanished, the user would read a sentence that was supposed to be sourced and see nothing. The gates measure this as UI dropout with a required rate of zero.

F22. How does a click become a highlight? The click sets the active citation; the drawer opens; regions come from the stream or from `GET /documents/{id}/chunks/{chunk_id}/evidence`; PDF.js renders the page to canvas; `overlayRectsForPage` converts PDF points to canvas pixels; overlay divs render with `mix-blend-multiply` so the text stays readable.

F23. What coordinate space are the boxes in? PyMuPDF page space — origin top-left, units in PDF points. The stored page width and height are what allow scaling to the rendered canvas.

F24. Why cache quote regions? Resolving a quote to coordinates involves fuzzy alignment over page spans, and the same quote is resolved repeatedly — during finalisation, when the drawer opens, and again if the user reopens it. The cache turns that into one computation. The measured cache hit rate is 100 percent.

F25. What is the evidence trace? A structured record per claim: retrieval chunk, selected chunk, whether rebinding happened, candidates searched, localisation status, support status, UI status, and whether it survived into the final citations.

F26. Does the trace log document text? No. Identifiers, statuses, and scores only. You can debug a citation without user document content flowing into logs.

F27. Summarise the anti-hallucination stack. Twelve deterministic layers: prompt rules, scope restriction, relevance gating, mid-stream marker stripping, answer formatting, refusal verification, quote verification, claim rebinding, never-invented geometry, visual content policy, a persistence gate, and visibility rules. Each is separately testable.

G. Auth, quotas, and privacy

G1. How does the server know who I am without a login? The browser generates a UUID, stores it in localStorage as `docusage_guest_session`, and sends it as the `X-Guest-Session` header.

G2. What happens when I sign in? Supabase Google OAuth returns an access token, which becomes an `Authorization: Bearer` header and replaces the guest header.

G3. What if both headers are present? The bearer token wins. This handles the real case where a user signs in but the browser still holds the trial identifier. Without the rule, the same person could be treated as two different actors depending on request ordering.

G4. Are passwords stored? None. Google handles authentication; the backend only verifies a JWT signature and stores the subject claim.

G5. Which OAuth providers are supported? Google only, deliberately — it keeps the sign-in screen to one button.

G6. What are the exact limits? Guest: 1 PDF, 15 questions for the whole trial. Signed in: 5 PDFs, 100 questions per calendar month. Upload limit 25 MB.

G7. Can limits be changed without redeploying code? Yes. They are environment variables read at request time. Edit `.env`, restart.

G8. Where are quotas checked? In the API shell before any retrieval starts, so a blocked request costs zero embedding compute and zero LLM tokens.

G9. Does deleting a PDF free a slot? Yes, immediately. `pdfs_used` counts live rows rather than reading an incrementing counter, so it cannot drift out of sync with reality.

G10. When is a question counted? After retrieval succeeds, not when the request arrives. A request that fails during retrieval does not consume your allowance.

G11. Does regenerating cost a question? No. `if not request.regenerate: check_question_allowed(...)` — you already paid for that answer.

G12. What does a 402 response contain? A `QUOTA_EXCEEDED` body with the resource, the limit, the amount used, and an `upgrade_hint` of `sign_in`, `delete_document`, or `wait_for_reset` — so the interface never has to guess by parsing prose.

G13. Can guest mode be turned off? Yes, `GUEST_TRIAL_ENABLED=false`, without touching route code.

G14. What happens to documents that predate ownership columns? They have a NULL owner and stay readable. Documented and intentional, so that upgrading an existing installation does not orphan data.

G15. Can another user read my conversation? No. `assert_conversation_owner` runs on every conversation route, including before the chat stream opens.

G16. Does my document text go into logs? No. The evidence trace records identifiers and statuses only, and LLM errors are run through a redaction layer whose stated policy is that messages must never include API keys or auth headers.

G17. Where is my PDF stored? On the API server's disk under `./data`, bind-mounted from the host so container rebuilds do not lose it. It is served back only to its owner through `/documents/{id}/file`.

H. Evaluation

H1. How do you know the system works? Seven layers: 43 Python test files and 5 frontend test files, a 45-question baseline, a highlight eval, 20 held-out real-document cases, a 186-test adversarial pack, machine-checked acceptance gates, and a launch audit.

H2. What does the baseline measure? Retrieval hit at K 0.90, page hit 0.825, document hit 1.0, MRR 0.785, retrieval latency 27 ms mean; answer keyword hit 0.975, citation page hit 0.875, and unanswerable handling 1.0 with a 1.0 refusal rate.

H3. Which numbers matter most there? The unanswerable rows. A perfect refusal rate on questions the corpus does not answer is the metric that most directly reflects the project's thesis.

H4. What is the worst category in the baseline? Conversational, at 0.333 retrieval hit against 0.9 to 1.0 elsewhere. Query rewriting helps but does not solve follow-up turns. That is the honest answer to "where does it struggle?"

H5. What does the highlight eval report? Claim verification, localisation, and highlight precision all 1.0; unresolved rate 0.0; cache hit rate 1.0; fallback rate 0.222.

H6. Is a 22 percent fallback rate bad? No — that is the system correctly choosing a page reference over a fake sentence highlight. Reading it as a defect reads the design backwards.

H7. What is the held-out evaluation? Twenty adjudicated cases against a generated operations handbook the pipeline was not tuned on — 10 native-text localisation, 10 retrieval. It indexes into an isolated store so evaluation never touches the live corpus.

H8. Why isolate the held-out index? Two reasons. Evaluation must not pollute a user's searchable corpus, and results must not depend on whatever documents happen to be indexed at the time.

H9. What does the adversarial pack cover? 186 tests: the same question asked three times for stability, paraphrases, wrong-document scope, invented markers, visual false-precision, and refusals on unrelated questions — plus fifteen regression modules. It calls no live LLM.

H10. What are the acceptance bars? Adversarial 100 percent; retrieval hit at least 92 percent; context recall at least 90 percent; grounding at least 90 percent; localisation at least 95 percent; wrong page at most 2 percent; false precise at most 1 percent; UI dropout 0; rerank zero-result 0; instability at most 5 percent; at least 20 held-out cases with at least 10 native and 10 retrieval.

H11. What does "fails closed" mean, and why does it matter? An empty case catalogue would give 0 percent wrong pages, which passes every rate gate vacuously while testing nothing. So count gates must pass before rate gates are considered. The module states it directly: *no vacuous 0% wrong-page pass*.

H12. Is there an example of that in the repository? Yes — `real_document_latest.json` currently shows `case_count: 0`. Under a naive gate that file would look perfect. Under this design it certifies nothing, which is exactly why the count floors exist.

H13. What is the difference between the CI and release profiles? `--profile ci` is the engineering gate: the adversarial pack plus the rerank invariant. `--profile release` is the ship stamp and enforces the held-out count floors.

H14. What does the launch audit add? Nine production scenarios, a stability check, latency measurement, and four failure-injection probes. Latest run: 9 of 9 scenarios, stable across three runs, p50 626 ms, p95 1105 ms.

H15. What are the failure-injection probes? Ghost document isolation, Super Focused aborting on no selection, an empty allowlist dropping hits rather than searching all of Chroma, and invented markers being stripped. All four pass.

H16. Why is `launch_approved` false when everything passes? Because `--signoff` was not given. Engineering passing and a launch being approved are deliberately different things. Tests prove the properties you thought to test still hold; they cannot prove you thought of everything.

H17. Do the evaluation suites call a live LLM? The adversarial pack and acceptance gates do not, which is what makes them fast and deterministic. The full baseline does when generation is enabled, and it can be run with `--skip-generation`.

I. Interview and viva questions

I1. Walk me through one question end to end. Identity resolves from the header or token. Ownership and quota checks pass. The document scope narrows to ready documents you own. Turn analysis classifies intent and decides whether a rewrite is needed. An answer plan picks the briefing components. Dense and BM25 retrieval each return 20 candidates, fused with RRF, topped up with any reserved query-type slots, and reranked by a cross-encoder. Surviving chunks become the recall pool E1 to En, and the citeable subset is determined by the relevance floors. A grounded prompt is built with the full passage text. The LLM streams while markers are resolved incrementally. Afterwards the refusal is verified, every quote is checked

against its chunk, every claim is re-scored across the pool and rebound if a better passage wins by a clear margin, winners are mapped to PDF bounding boxes, and the interface is told exactly which citations it may present and how precisely.

I2. What is the hardest problem you solved? Mapping a paraphrased claim back to specific coordinates in a PDF. The model paraphrases; PDF text extraction is messy with ligatures, hyphenation, and irregular whitespace; and chunks cross page boundaries. It needed a four-level alignment cascade at index time, a claim-scoring layer to pick the right passage, neighbour stitching for boundary-spanning claims, and a strict rule that geometry is never invented.

I3. What are you most proud of? That the system refuses to fake precision. Every layer had an easier option — show the whole chunk highlighted, trust the model's quote, cite whatever ranked first — and each was rejected in favour of a check. The evaluation harness that fails closed is the same instinct applied to the process.

I4. How is this different from a naive RAG tutorial? A tutorial is: chunk, embed, retrieve top-k, stuff into a prompt, return the answer. This adds hybrid retrieval with rank fusion, cross-encoder reranking, calibrated score floors, query-type profiles, deterministic query understanding, index-time coordinate mapping, quote verification, claim-level rebinding, a refusal verifier, a visual content policy, and an evaluation suite with gates that fail closed. The tutorial trusts the model; this treats its output as untrusted input.

I5. Where would it break under load? SQLite is a single writer, so concurrent uploads would contend. The embedding and reranker models are loaded in-process, so memory scales with worker count. Chroma is embedded, so it cannot be scaled independently of the API. The first fix is Postgres; the second is moving the models behind a shared inference service.

I6. Why BGE-small rather than a larger embedder? It runs on CPU in a few hundred milliseconds and fits a free-tier VM. More importantly, the architecture compensates: BM25 catches exact terms dense vectors blur, and the cross-encoder reranks whatever the first stage returns. Spending the compute budget on reranking rather than a bigger embedder is the better trade at this scale.

I7. If you had to remove one component, which and why? The Streamlit prototype, because it is dead weight. Among live components, the answer-plan sub-queries are the least load-bearing — they add latency for a bounded recall gain. The reranker would be the most damaging to remove.

I8. What would you do differently starting over? Build the evaluation harness first. The scoring constants were tuned as problems appeared; with the held-out catalogue in place from day one they could have been tuned against a fixed target from the start. I would also design the evidence sidecar before the chunking strategy, since chunking choices constrain what geometry you can recover afterwards.

I9. How do you know the citations are correct rather than plausible? Because correctness is machine-checked, not asserted. The held-out suite measures localisation, wrong-page, and false-precise rates on documents the pipeline was not tuned on, and the acceptance gates refuse to certify unless the counts and rates both clear their bars.

I10. What is the biggest risk in the design? That the validation layers are individually correct but collectively too strict, so useful evidence is suppressed and the product feels unhelpful. That is exactly what

the recall-fallback path and the citeable-versus-page-only split exist to balance, and why the fallback rate is measured rather than ignored.

I11. Why so many configuration constants? Because they encode measured decisions rather than guesses, and putting them in one file with their reasoning in comments makes them auditable and retunable. The MiniLM offset comment is the clearest example: it records the true positive and true negative that produced the value.

I12. How long did this take, and how did it evolve? Thirty-one commits across the arc visible in the history: an initial Streamlit knowledge assistant, a move to Next.js, cross-encoder reranking, three iterations of evidence-based highlighted citations, then the platform layer for guest trials and Google auth.

I13. What does the code review of your own project find? Three things. `COLLECTION_NAME` still defaults to `ml_notes`, a leftover from the first version. The Streamlit prototype should be removed or clearly archived. And `.env.example` disagrees with `config.py` on `TOP_K` (3 versus 5), which should be reconciled.

I14. Is agentic mode implemented? No. It is a seam, a disabled flag, and a tool-name constant. The source says plainly: *do not pretend this module is autonomous*.

I15. Why does that honesty matter? Because a reviewer will find it either way. Volunteering the boundary reads as judgement; being caught by it reads as overselling.

J. Operations

J1. How do I run it locally? Backend: create a virtualenv, `pip install -r requirements.txt`, copy `.env.example` to `.env`, run `uvicorn backend:app --reload`. Frontend: `npm install`, copy `.env.example` to `.env.local`, `npm run dev`. Open `http://localhost:3000`.

J2. Why is the first request so slow? It downloads the embedding and reranker model weights — minutes, not seconds. Say this before a demo so nobody thinks it has hung.

J3. Do I need API keys to try it? For retrieval, no. For generation you need at least one provider key, or Ollama running locally. Sign-in is optional; without Supabase credentials the app runs guest-only.

J4. What are the most important environment variables? `GROQ_API_KEY`, `CORS_ORIGINS`, `DATABASE_URL`, `AUTH_PROVIDER`, `SUPABASE_JWT_SECRET`, and the four quota variables. On the frontend, `NEXT_PUBLIC_API_URL`.

J5. How is it deployed? Frontend on Vercel with the root directory set to `frontend`. API in Docker on a single small VM with `./data`, `./chroma_db`, and the model cache mounted.

J6. Why bind-mount those directories? So a container rebuild or VM restart never loses a user's PDFs, embeddings, or accounts. The model cache is mounted so first boot does not re-download weights.

J7. Why does the health check have a 300-second start period? Because first boot downloads models. Without the grace period the container would be marked unhealthy and restarted in a loop before it could finish.

J8. What happens if I delete `chroma_db` ? You lose the embeddings and the searchable chunk text. It is recoverable by re-indexing the PDFs, since the originals are on disk and the SQLite rows survive.

J9. What if I delete `data/documents.db` ? Unrecoverable. Metadata, conversations, evidence geometry, and accounts all live there.

J10. What if the BM25 index gets out of sync? It cannot stay out of sync. It is in-memory, built from the Chroma corpus, and fingerprint-invalidated on any change.

J11. What does `reconcile_index` do at startup? Sweeps up orphans that appeared while the process was down — for example if the container was killed mid-delete. If it fails, startup continues and logs the failure rather than refusing to boot.

J12. How do I switch the reranker? Set `RERANKER_MODEL` . Aliases `bge` and `minilm` are built in, and full HuggingFace identifiers pass through.

J13. How do I change the provider order? `LLM_PRIMARY_PROVIDER` and `LLM_FALLBACK_PROVIDERS` — the latter is a comma-separated list.

J14. Is there a mock mode for frontend work? Yes, `NEXT_PUBLIC_USE MOCK=true` returns stub documents and chat responses so the interface can be developed without a backend.

J15. How do I diagnose a document that will not index? Check the `index_error` column and the indexing logs, which record the engine used, how many pages had text, and character counts per engine. There is also a `scripts/diagnose_pdf.py` helper.

K. Quick reference card

Keep this to hand when presenting.

The pitch: Most document chatbots treat the answer as the product. DocuSage treats verification as the product.

The stack: Next.js and PDF.js on Vercel; FastAPI in Docker; ChromaDB plus BM25 plus a BGE cross-encoder; Groq for generation; SQLite; Supabase Google auth.

The numbers that matter:

Thing	Value
Chunk size and overlap	1200 / 150 characters
Candidates per retriever	20
RRF constant	60
Final evidence slots	5
Recall floor / citation floor	25 / 30
Claim rebind margin / support floor	0.12 / 0.42
Claim score weights	0.40 span, 0.45 coverage, 0.15 relevance
Memory window	6 messages
Guest limits	1 PDF, 15 questions
Free tier limits	5 PDFs, 100 questions per month
Adversarial pack	186 tests, 100 percent required
Held-out cases	20 (18 native, 10 retrieval)
Localisation / grounding / retrieval hit	100% / 100% / 100%
Latency p50 / p95	627 ms / 1105 ms

The three sentences to say if you only get three:

1. Every claim carries a citation that resolves to a highlighted sentence in the original PDF.
2. The model's output is treated as untrusted — quotes are verified against the retrieved text, claims are rebound to the passage that actually supports them, and anything that cannot be traced is dropped rather than displayed.
3. The quality bars are machine-checked and fail closed, so an empty test catalogue cannot certify a release.

End of handbook. Written from the DocuSage repository as it stands, for Atif Saeed.